

## **CHAPTER III**

### **FRAMEWORK AND METHODOLOGY**

Chapter 3 presents the framework and methodology used to design, develop, and evaluate the proposed blockchain-enabled LoRaWAN agricultural monitoring prototype. It discusses the study's conceptual framework, research design, prototype architecture and experimental setting, development and data collection procedures, security boundary and threat model, evaluation methods, data analysis, and ethical considerations. Together, these sections explain how the prototype will be developed and systematically tested under normal, connectivity-interruption, and selected security-attack conditions.

#### **3.1 Conceptual Framework**

The Input–Process–Output (IPO) model will be used as the conceptual framework because the study focuses on the design, operation, and evaluation of a technological prototype. The inputs consist of agricultural sensor data, LoRaWAN network events, system loads, and normal and attack test conditions. These inputs are processed through LoRaWAN transmission, Raspberry Pi edge computing, ChirpStack, MQTT, Node-RED, PostgreSQL, Hyperledger Fabric verification, and Grafana monitoring. The resulting outputs will be evaluated in terms of system performance, security effectiveness, data integrity, traceability, and resilience. The IPO framework therefore provides a clear and logical representation of how the system receives data, processes and protects them, and produces measurable outcomes.

As shown in Figure 3.2, the input stage will consist of agricultural sensor data, LoRaWAN network events, and predefined test conditions. The sensors will generate environmental and soil measurements, while the network events will represent normal transmissions, connectivity interruptions, unauthorized requests, replay attempts, and altered data. Normal and attack conditions will be included so that the prototype can be evaluated during regular operation and under selected security threats. These inputs will represent the data, communication behavior, and operating conditions that the proposed agricultural monitoring system will be expected to handle.

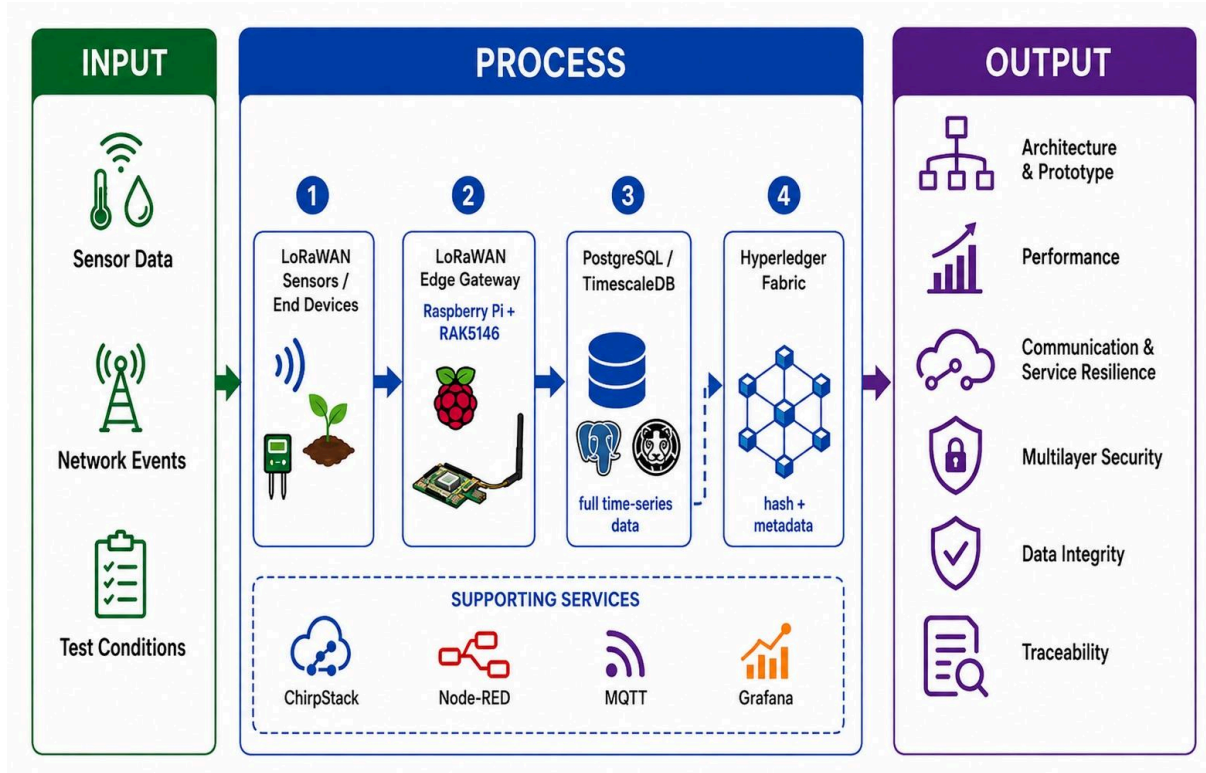


Figure 3.1. Conceptual Framework of the Study

In the process stage, sensor readings will first be transmitted through LoRaWAN, which will provide long-range communication with low power consumption and will therefore be suitable for agricultural areas. A Raspberry Pi will function as the edge gateway and will run ChirpStack, MQTT, Node-RED, and local processing services. ChirpStack will manage LoRaWAN devices and packets, MQTT will support lightweight data exchange, and Node-RED will control data flow, validation, alerts, and automation. PostgreSQL will store structured information such as devices, locations, users, and alerts. Grafana will retrieve the stored readings and system metrics to provide real-time dashboards and monitoring. Hyperledger Fabric will record the hashes and essential metadata of the sensor records, rather than the complete readings, to support permissioned access, integrity verification, traceability, and auditability without placing excessive storage demands on the blockchain.

The output stage will produce a resilient blockchain-enabled LoRaWAN–Edge IoT architecture and a working prototype for remote data acquisition, edge processing,

blockchain-based data recording, authentication, and secure data exchange. Its performance will be evaluated at the end-to-end system and blockchain levels using packet-delivery rate, latency, throughput, transaction success rate, resource utilization, and blockchain overhead under normal operation and controlled transaction workloads. Its security and resilience will then be assessed under communication and service failures, varying traffic and transaction loads, and multilayer security threats. Data integrity and traceability will also be evaluated across the application, storage, and blockchain layers through verifiable records of data origin, processing, storage, blockchain transactions, and chronological history.

## **3.2 Methodology**

This study will adopt the Design and Development Research methodology because it will provide a systematic approach to designing, developing, evaluating, and improving a technological prototype. The analysis phase will identify the agricultural monitoring needs and system gaps to be addressed. The design phase will develop the resilient blockchain-enabled LoRaWAN–Edge IoT architecture, while the development phase will build and integrate the prototype for remote data acquisition, edge processing, blockchain-based recording, authentication, and secure data exchange. The evaluation and validation phase will measure end-to-end and blockchain performance and assess the prototype’s security, data integrity, traceability, and resilience under normal operation, controlled workloads, and simulated adverse conditions. Finally, the revision phase will use the documented evaluation results to correct technical issues and refine the prototype.

### **3.2.1 Analysis and Needs Assessment**

The Analysis and Needs Assessment phase will examine the agricultural monitoring process and stakeholder needs to identify the system’s functional and non-functional requirements. These requirements will then be consolidated into architectural requirements that will guide the Design phase.

### 3.2.1.1 Domain and Process Analysis

Consistent with ISO/IEC/IEEE 29148:2018 [94], system requirements should be derived from an understanding of the operational environment, stakeholder needs, existing processes, and system constraints. Accordingly, the domain analysis will examine the agricultural monitoring environment, including its needs, actors, operational conditions, and problems, while the process analysis will examine how agricultural data are collected, transmitted, processed, stored, viewed, and verified. The findings will provide the basis for defining the functional and non-functional requirements of the proposed system.

Based on the literature synthesized in Section 2.1.11, the domain analysis identified the following agricultural monitoring needs:

- **Wide and remote monitoring areas.** Agricultural fields may extend across distant locations, requiring sensor readings to travel over long distances while consuming minimal power.
- **Affordable operation.** Farms may have limited financial and computing resources, making a cost-conscious monitoring system necessary.
- **Continuity during Internet interruptions.** Internet service may be unstable in remote farms; therefore, monitoring data must remain available and should not be lost during outages.
- **Management of continuous sensor data.** Agricultural sensors generate time-stamped readings continuously, which must be received, processed, stored, and viewed chronologically.
- **Trustworthy agricultural records.** Because sensor readings may support agricultural decisions, stored records must remain accurate, verifiable, and traceable.
- **Secure data exchange.** Only authorized devices and users should be allowed to submit, access, or modify agricultural monitoring records.

Based on the agricultural monitoring processes and requirements synthesized in Sections 2.1.2–2.1.7 and 2.1.11, the process analysis will examine

how data must move from the field to the intended users while remaining available and trustworthy. Because agricultural areas may be vast, dispersed, and remotely located, soil and environmental conditions must first be captured near their source and transmitted over the required field distance to a processing point. Since readings will be generated continuously, they must then be validated, timestamped, and organized before being stored as chronological records. These records must remain accessible for monitoring and decision-making, even when connectivity is unstable, while their integrity, origin, and history must be verifiable.

Accordingly, the monitoring process will involve six connected phases: collection of soil and environmental readings; transmission from the field to a processing point; processing through validation, timestamping, and organization; storage of complete chronological records; viewing of current and historical information; and verification of record integrity, origin, and traceable history. Analyzing the needs and dependencies at each phase will provide the basis for selecting appropriate technologies and developing the architecture during the Design phase. The complete process is shown in Figure 3.2, which is developed by the researcher based on the literature synthesized in Sections 2.1.2–2.1.7 and 2.1.11.

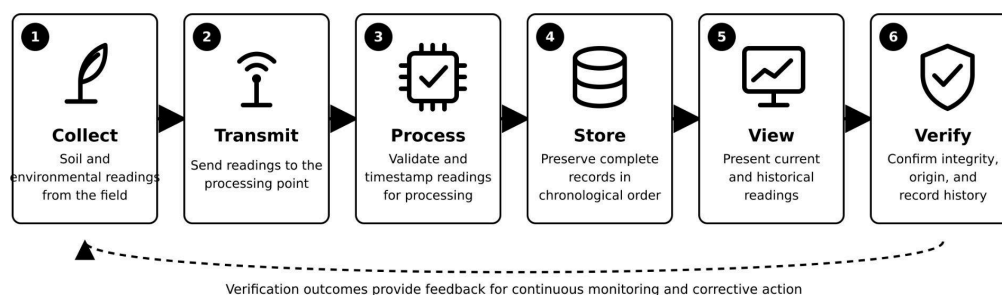


Figure 3.2. Agricultural Monitoring Data Process

### 3.2.1.2 Stakeholder Analysis

The stakeholder analysis will identify the individuals who will operate, use, manage, or evaluate the agricultural monitoring prototype. Because this is a prototype study, the researcher will serve as its primary operator, while other stakeholders will have access appropriate to their roles and technical capabilities.

- **Researcher:** Will install and operate the prototype, collect data, conduct the tests, and access all system records and performance results.
- **Farm personnel:** Will require simple access to current sensor readings and alerts without needing advanced knowledge of LoRaWAN or blockchain.
- **Farm managers:** Will need to view dashboards and historical readings to support monitoring and agricultural decisions.
- **System administrator:** Will manage devices, user accounts, software services, security configurations, backups, and system recovery.
- **Evaluator or auditor:** Will require read-only access to logs, hashes, transaction records, and data histories to verify integrity and traceability.

These stakeholder needs, responsibilities, access levels, and technical capabilities will guide the system's functional, security, access-control, and usability requirements.

### 3.2.1.3 Functional Requirements

Functional requirements define the specific tasks and operations that the system must perform. Based on the identified agricultural monitoring needs, the required flow of data through the six process phases, and the responsibilities and access needs of the stakeholders, the proposed system shall satisfy the following functional requirements:

- **FR-01 Data collection:** The system shall collect soil and environmental readings from agricultural sensors.
- **FR-02 Data transmission:** The system shall transmit sensor readings from field devices to the edge gateway.

- **FR-03 Data processing:** The system shall validate, timestamp, organize, and route each received reading.
- **FR-04 Data storage:** The system shall store complete sensor readings in chronological order for retrieval and analysis.
- **FR-05 Authentication and authorization:** The system shall authenticate devices and users and restrict actions according to their assigned permissions.
- **FR-06 Integrity verification:** The system shall generate a hash for each accepted record and compare it with the corresponding blockchain record to detect changes.
- **FR-07 Monitoring:** The system shall display current and historical sensor readings, alerts, and operational status.
- **FR-08 Offline operation:** The system shall continue receiving, processing, and storing readings during Internet or blockchain-service interruptions.
- **FR-09 Recovery and reconciliation:** The system shall retain pending records and submit or reconcile them after the interrupted service is restored.
- **FR-10 Traceability:** The system shall maintain verifiable information about each record's origin, processing, storage, and blockchain transaction history.

#### 3.2.1.4 Non-Functional Requirements

Non-functional requirements define the quality and operating conditions under which the system must perform its functions. The domain analysis will establish the requirements for affordability, continuity, and operation within limited resources; the process analysis will establish the required performance, reliability, scalability, resilience, and data integrity across the complete data flow; and the stakeholder analysis will establish security and usability requirements. Based on the domain, process, and stakeholder analyses, the proposed system shall satisfy the following non-functional requirements:

- **NFR-01 Performance:** The system shall process and deliver sensor and blockchain transactions with measurable packet delivery, latency, throughput, and transaction success.

- **NFR-02 Reliability:** The system shall consistently collect, process, and store legitimate sensor readings without unintended loss or duplication.
- **NFR-03 Security:** The system shall resist unauthorized access, replay, spoofing, flooding, and unauthorized data modification.
- **NFR-04 Resilience:** The system shall maintain essential local operations during Internet or Fabric unavailability and recover pending records after service restoration.
- **NFR-05 Resource efficiency:** The system shall operate within the CPU, memory, storage, and network capacity of the edge gateway.
- **NFR-06 Scalability:** The system shall accommodate varying sensor-data and blockchain-transaction workloads without unacceptable performance degradation.
- **NFR-07 Data integrity:** The system shall preserve the accuracy and consistency of records across the application, database, and blockchain layers.
- **NFR-08 Usability:** The system shall present sensor readings, alerts, and system status through an accessible monitoring dashboard.
- **NFR-09 Cost:** The system shall use cost-conscious hardware and open-source software appropriate for prototype-scale agricultural monitoring.

### 3.2.1.5 Consolidated Architectural Requirements

The domain, process and stakeholder needs were consolidated with the functional and non-functional requirements to establish the following architectural implications:

- **Wide and remote monitoring areas** → FR-01, FR-02, NFR-01, and NFR-05 → The architecture will require a low-power, long-range communication layer for transmitting field readings.
- **Intermittent Internet and service availability** → FR-08, FR-09, and NFR-04 → The architecture will require local processing and storage, pending-record retention, and recovery after service restoration.

- **Continuous sensor-data generation** → FR-03, FR-04, NFR-01, and NFR-06 → The architecture will require an efficient data-processing flow and scalable time-series storage.
- **Trusted and verifiable records** → FR-06, FR-10, and NFR-07 → The architecture will require tamper-evident verification and a traceable history for every sensor record.
- **Authorized and secure data exchange** → FR-05 and NFR-03 → The architecture will require authentication, authorization, and access control across its connected layers.
- **Stakeholder monitoring and usability needs** → FR-07 and NFR-08 → The architecture will require accessible dashboards, alerts, and role-appropriate access to monitoring information.
- **Limited financial and computing resources** → NFR-05 and NFR-09 → The architecture will require resource-efficient, cost-conscious, and preferably open-source components.
- **Complete agricultural monitoring process** → FR-01, FR-02, FR-03, FR-04, FR-06, FR-07, FR-10, NFR-02, and NFR-07 → The architecture will require an integrated data flow that supports the collection, transmission, processing, storage, viewing, and verification of agricultural monitoring records.

These consolidated requirements will guide the selection and integration of the specific technologies presented in the Design phase.

### 3.2.2 Design

The Design phase will translate the requirements identified during the domain, process, and stakeholder analyses into the proposed system architecture. As shown in Table 3.1, each agricultural monitoring need will be matched with the appropriate functional and non-functional requirements and corresponding technological components, providing a clear basis for how the prototype will be structured and integrated.

Table 3.1. Consolidated Requirements and Corresponding Architectural Design

| Domain, process or stakeholder need          | Related requirements                             | Architectural requirement                                                                                                 | Proposed design response and selected components                                                                                                                                                                                                                                                                                                                |
|----------------------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Complete agricultural monitoring process     | FR-01–FR-04, FR-06, FR-07, FR-10; NFR-02, NFR-07 | Integrated end-to-end flow for collecting, transmitting, processing, storing, viewing, and verifying agricultural records | Agricultural sensors and RAK4631 modules will collect readings; LoRaWAN and the RAK5146 gateway will transmit them; ChirpStack, MQTT, and Node-RED will receive and process them; PostgreSQL/TimescaleDB will store complete records; Grafana will display monitoring information; and Hyperledger Fabric will support integrity verification and traceability. |
| Wide and remote monitoring areas             | FR-01, FR-02; NFR-01, NFR-05                     | Low-power, long-range field communication                                                                                 | Soil-moisture, UV-light, barometric-pressure, ambient-light, temperature, humidity, and rain sensors connected to RAK4631 modules will transmit readings through LoRaWAN AS923 to the RAK5146 gateway.                                                                                                                                                          |
| Limited financial and computing resources    | NFR-05, NFR-09                                   | Cost-conscious and resource-efficient operation                                                                           | A Raspberry Pi 4 and open-source services will provide the edge-computing platform.                                                                                                                                                                                                                                                                             |
| Intermittent Internet or Fabric availability | FR-08, FR-09; NFR-04                             | Continued local operation, pending-record retention, and recovery                                                         | Locally hosted ChirpStack, MQTT, Node-RED, PostgreSQL/TimescaleDB, and Fabric services will support offline operation and subsequent reconciliation.                                                                                                                                                                                                            |
| Continuous sensor-data generation            | FR-03, FR-04; NFR-01, NFR-06                     | Continuous processing and scalable time-series storage                                                                    | MQTT will receive the data stream, Node-RED will validate and route readings, and PostgreSQL/TimescaleDB will store complete time-series records.                                                                                                                                                                                                               |
| Trusted and verifiable agricultural records  | FR-06, FR-10; NFR-07                             | Tamper-evident verification and record traceability                                                                       | Node-RED will generate SHA-256 hashes, while Hyperledger Fabric will record the hashes, metadata, and transaction identifiers.                                                                                                                                                                                                                                  |
| Authorized and secure data exchange          | FR-05; NFR-03                                    | Authentication, authorization, and access control across system layers                                                    | LoRaWAN device authentication, MQTT credentials, and Fabric identities and endorsement policies will restrict participation.                                                                                                                                                                                                                                    |
| Accessible monitoring information            | FR-07; NFR-08                                    | Dashboard-based visualization of readings, alerts, and system status                                                      | Grafana will display current and historical readings, alerts, and system-performance information.                                                                                                                                                                                                                                                               |

Note: FR= Functional requirements, NFR= Non-functional requirements

Based on the requirements established during the Analysis phase and the supporting literature reviewed in Chapter 2, each component will be selected for a specific architectural role. LoRaWAN will support low-power, long-range field communication; Raspberry Pi and locally hosted services will enable affordable edge operation during Internet interruptions; PostgreSQL/TimescaleDB will manage complete chronological sensor records; and Hyperledger Fabric will provide permissioned identity management, tamper-evident verification, and traceability. MQTT and Node-RED will support lightweight data exchange and processing, while Grafana will provide accessible monitoring of current and historical information.

The architecture will begin with agricultural sensors that will collect soil moisture, ultraviolet light, barometric pressure, ambient light, temperature, humidity, and rainfall data. The experimental testbed will use multiple agricultural sensors to collect soil and environmental data. As shown in Figure 3.3, these will include soil-moisture probes and sensors for ultraviolet light, barometric pressure, ambient light, temperature, humidity, and rainfall. The RAK4631 core module will process and transmit the readings through LoRaWAN, while the baseboards will connect and power the sensor modules. RS485, 4–20 mA, and SDI-12 interfaces will support the future integration of additional agricultural sensors. Powered by solar-charged batteries, the sensor nodes will use RAK4631 modules to transmit readings through LoRaWAN, which is suited to low-power, long-range communication across wide agricultural areas. The RAK5146 gateway, configured for the Philippine AS923 frequency plan, will receive the transmissions and connect the field sensors to the edge environment.

At the edge, the Raspberry Pi 4 will provide an affordable and resource-conscious platform for hosting the system services locally. ChirpStack will manage LoRaWAN devices and packets, while MQTT will support lightweight data exchange. Node-RED will validate, timestamp, and route the readings through the processing flow. Local hosting will allow essential data

collection and processing to continue even when Internet connectivity becomes unavailable.

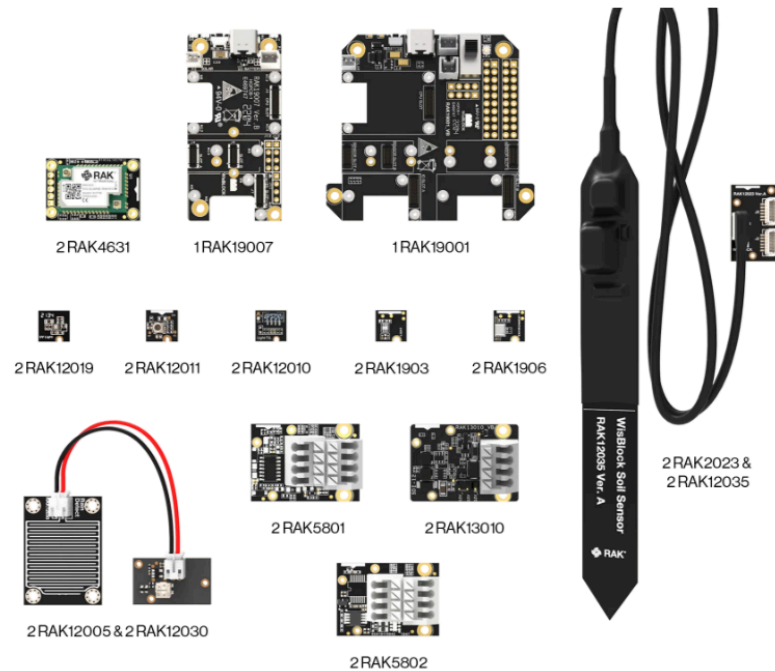


Figure 3.3. Agricultural Sensors and IoT Components Used in the Prototype

Figure 3.4 shows the Raspberry Pi–RAK5146 LoRaWAN edge gateway that will be used in the prototype. The gateway will be assembled by mounting the RAK5146 concentrator on its Pi HAT and connecting it to the Raspberry Pi 4, LoRa antenna, and optional GPS antenna. Raspberry Pi OS Lite will be installed on a microSD card, after which SSH will be configured, root login disabled, and the firewall enabled. Docker and the required Raspberry Pi interfaces—SPI, I<sup>2</sup>C, and the serial port—will then be configured. The gateway EUI will be generated from the Raspberry Pi network interface, while the LoRa gateway software will forward uplink and downlink packets to ChirpStack. Because the prototype will be deployed in the Philippines, the gateway will use the AS923 frequency plan rather than the European 868 MHz configuration shown in the reference setup.

PostgreSQL/TimescaleDB will store the complete sensor readings because agricultural data will be generated continuously and organized according to time. Hyperledger Fabric will complement the operational database by recording only the SHA-256 hashes and selected metadata of each reading. Its permissioned identities and endorsement policies will restrict blockchain participation, while hashes and transaction identifiers will support integrity verification and traceability without placing the complete sensor dataset on the ledger.

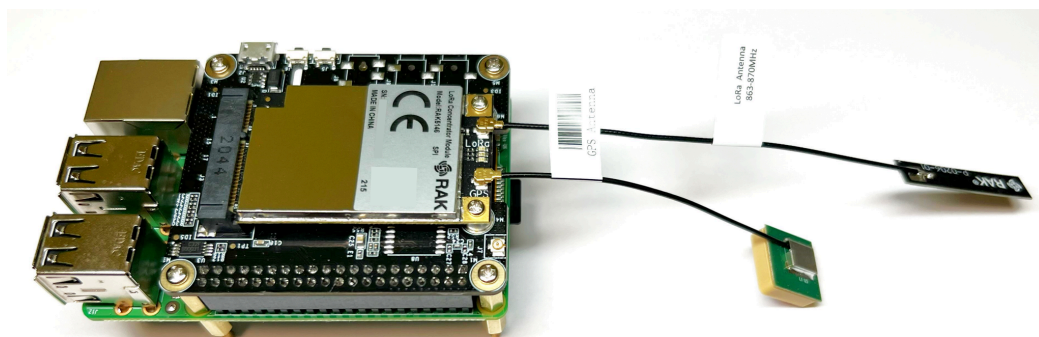


Figure 3.4. Raspberry Pi-Based LoRaWAN Edge Gateway Used in the Prototype

Finally, Grafana will present the stored readings and system metrics through accessible dashboards showing current conditions, historical trends, alerts, and system status. Together, these components will establish an integrated flow from field-data collection to processing, storage, verification, and monitoring while supporting the identified performance, security, resilience, integrity, and traceability requirements. Figure 3.5. presents the overall architecture of the proposed LoRaWAN–edge–blockchain prototype.

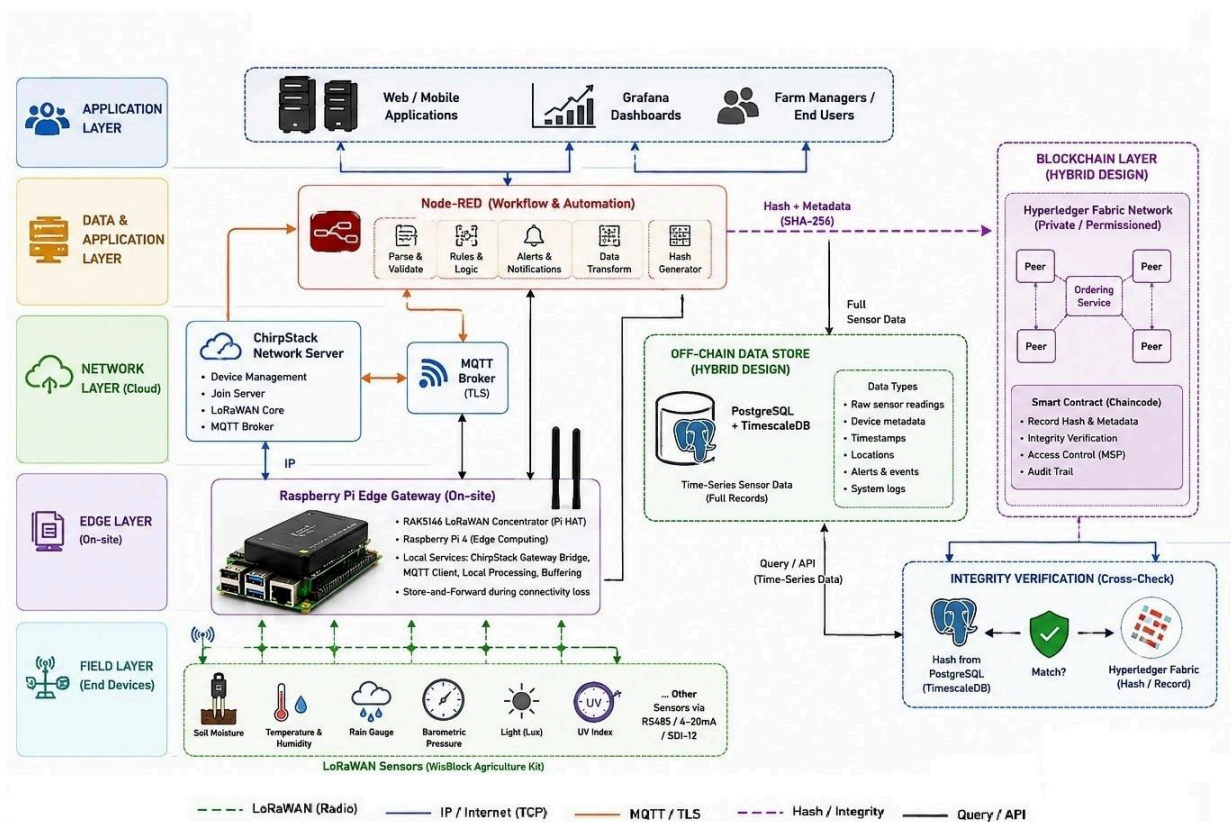


Figure 3.5. Proposed LoRaWAN-Edge-Blockchain Architecture for Secure Agricultural Monitoring

### 3.2.3 Development

The prototype will be developed in stages, beginning with the assembly and functional checking of the agricultural sensor nodes and Raspberry Pi-RAK5146 LoRaWAN gateway. The required software services—ChirpStack, Mosquitto MQTT, Node-RED, PostgreSQL/TimescaleDB, Grafana, and Hyperledger Fabric—will then be installed, configured, and integrated. Preliminary checks will confirm that sensor readings can travel through the complete system, from field transmission and edge processing to storage, visualization, and blockchain recording, before formal evaluation begins.

ChirpStack will be deployed locally on the Raspberry Pi through Docker Compose and configured for the AS923 frequency plan. The RAK5146 concentrator will

forward LoRaWAN packets to the local ChirpStack Gateway Bridge, while the gateway and sensor nodes will be registered using their required identifiers and activation keys. ChirpStack will publish decoded readings as JSON messages to the locally hosted Mosquitto MQTT broker. Node-RED will subscribe to the corresponding topics, validate and timestamp the readings, generate alerts when required, and route the processed data to the storage and verification layers. This local setup will allow data collection and processing without dependence on a remote cloud platform.

PostgreSQL with the TimescaleDB extension will serve as the operational database for complete sensor readings, device information, timestamps, locations, and system events. TimescaleDB hypertables will organize the growing time-series dataset, while Grafana will connect through a restricted read-only account to display sensor readings and system indicators. Hyperledger Fabric will be deployed through Docker as a permissioned network, with identities and access policies controlling participation. The services will communicate through a private Docker and local-area network using defined service names, credentials, and required ports.

As each reading reaches the edge, Node-RED will convert it into a standard format, generate its SHA-256 hash, and store the complete record in PostgreSQL/TimescaleDB. The corresponding hash, device identity, timestamp, and selected metadata will then be submitted to the Hyperledger Fabric chaincode. Algorithm 1 presents this processing sequence from sensor-data preparation to blockchain anchoring.

---

**Algorithm 1. Secure Processing and Blockchain Anchoring of LoRaWAN Sensor Records**

---

**Input:** Decoded sensor reading (D) received from ChirpStack through MQTT

**Output:** Sensor record stored in PostgreSQL/TimescaleDB and corresponding integrity record submitted to Hyperledger Fabric

1. Receive decoded sensor reading (D) from MQTT.
2. Validate and process (D) in Node-RED.
3. **if** (D) is invalid **then**
4.     Reject the record and write the result to the system log.
5.     **Stop processing.**

6. **end if**
  7. Generate a unique trace\_id for the accepted sensor reading.
  8. Construct the canonical record (C) using the defined sensor fields in a fixed order and consistent format.
  9. Compute the record hash:
    - [
    - H\_C = SHA256(C)
    - ]
  10. Store the complete sensor record, trace\_id, and (H\_C) in PostgreSQL/TimescaleDB.
  11. Set the record's blockchain status to PENDING.
  12. Prepare the blockchain integrity record:
    - [
    - B = {trace\_id,\ H\_C,\ device\_id,\ timestamp,\ metadata}
    - ]
  13. Submit (B) to Hyperledger Fabric through the sensor-hash chaincode.
  14. **if** the Fabric transaction is successfully committed **then**
  15.     Obtain the Fabric transaction identifier (TxID).
  16.     Store (TxID) in the corresponding PostgreSQL/TimescaleDB record.
  17.     Change the blockchain status from PENDING to VERIFIED.
  18. **else**
  19.     Retain the record and its hash in PostgreSQL/TimescaleDB.
  20.     Keep the blockchain status as PENDING for later reconciliation.
  21. **end if**
  22. End processing for the sensor reading.
- 

### 3.2.4 Evaluation and Validation

This phase established how the completed prototype would be evaluated and validated. The performance plan covered the end-to-end system and blockchain layers using packet-delivery rate, latency, throughput, transaction success, resource utilization, and blockchain overhead. The validation covers the security and resilience plan, which includes communication and service failures, multilayer security, data integrity, and traceability.

#### 3.2.4.1. Performance Evaluation

The operation evaluation will measure the system performance. The test procedure will consist of two complementary tests. The End-to-End System

Performance Test will evaluate the overall prototype from sensor transmission to database storage and blockchain processing, while the Hyperledger Fabric Blockchain Performance and Overhead Test will separately assess the performance of the blockchain layer and the additional processing cost introduced by Fabric.

#### **3.2.4.1.1 End-to-End System Performance Test**

A normal-operation performance test will be conducted to establish the baseline performance of the prototype before the security and resilience tests are performed. During this test, only a registered agricultural sensor and authorized system components will be used; no invalid traffic, simulated attacks, Internet interruptions, or service failures will be introduced. The sensor will transmit one reading every 15 seconds for 30 minutes, producing approximately 120 expected transmissions per run, consistent with the baseline period used in the resilience experiment. The test will be repeated three times under the same configuration to obtain consistent baseline measurements.

During each run, the complete data flow from the sensor to the LoRaWAN gateway, ChirpStack, Mosquitto MQTT, Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric will be monitored. ChirpStack and sensor transmission records will be used to determine packet delivery; timestamps generated during sensor transmission and database storage will be used to measure end-to-end latency; Node-RED and Hyperledger Fabric logs will be used to determine blockchain transaction success; and PostgreSQL/TimescaleDB records will be used to determine the number of successfully processed sensor readings. Raspberry Pi CPU and memory utilization will be collected from system and Docker resource-monitoring logs throughout each run. The study already identifies system logs, sensor records, blockchain transactions, network measurements, and resource-usage data as the sources for subsequent performance analysis.

Table 3.2 organizes the baseline measures according to the main levels of the prototype. Packet-delivery rate assesses the reliability of LoRaWAN data acquisition, while latency and throughput describe the timeliness and processing capacity of the complete sensor-to-database flow. Transaction success rate confirms whether the corresponding hashes and metadata complete the blockchain-recording stage. CPU and memory utilization indicate whether the Raspberry Pi can support the integrated services within its available resources. Together, these measures establish the normal-operation baseline for comparison with the controlled blockchain workloads and subsequent security and resilience conditions.

Table 3.2. End-to-End Performance Measures

| System level           | Performance measure           | Rationale                                                                                       | Performance aspect measured                    |
|------------------------|-------------------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------|
| LoRaWAN communication  | Packet-delivery rate or PDR   | Determines whether legitimate sensor packets reliably reach ChirpStack                          | Measures remote-acquisition reliability        |
| End-to-end data flow   | End-to-end latency            | Measures the time required for a reading to travel from sensor transmission to database storage | Measures remote-acquisition timeliness         |
| End-to-end data flow   | Mean latency                  | Represents the average transmission-to-storage delay of successfully delivered readings         | Measures average transmission-to-storage delay |
| End-to-end processing  | System throughput             | Determines the rate at which legitimate readings are validated and stored                       | Measures data-processing capacity              |
| Blockchain integration | Transaction success rate      | Determines whether submitted hashes and metadata are successfully committed to Fabric           | Measures blockchain-recording reliability      |
| Edge platform          | Average CPU utilization or    | Measures the Raspberry Pi processing demand while the integrated services operate               | Measures edge-processing demand                |
| Edge platform          | Maximum CPU utilization or    | Identifies the highest Raspberry Pi processor demand during a run                               | Measures peak edge-processing demand           |
| Edge platform          | Average Memory utilization or | Measures the proportion of available Raspberry Pi memory                                        | Average edge-memory                            |

|               |                               |                                                                |                                  |
|---------------|-------------------------------|----------------------------------------------------------------|----------------------------------|
|               |                               | used by the system                                             | demand                           |
| Edge platform | Maximum memory utilization or | Identifies the highest Raspberry Pi memory demand during a run | Measures peak edge-memory demand |

The performance measures presented in Table 3.2 will be calculated using Equations (1)–(10). Packet-delivery rate determines whether legitimate sensor packets reliably reach ChirpStack will be computed using Equation (1).

$$PDR(\%) = \frac{N_{\text{received}}}{N_{\text{transmitted}}} \times 100 \quad (1)$$

Where: PDR = packet-delivery ratio;  $N_{\text{received}}$  = number of packets successfully received by ChirpStack;  $N_{\text{transmitted}}$  = total number of packets transmitted by the sensor nodes.

End-to-End Latency will be measures the time required for a reading to travel from sensor transmission to database storage and will be computed using Equation (2).

$$L_i = T_{\text{storage},i} - T_{\text{transmission},i} \quad (2)$$

Where:  $L_i$  = end-to-end latency of reading  $i$ ;  $T_{\text{storage},i}$  = time reading  $i$  is stored in the database;  $T_{\text{transmission},i}$  = time reading is transmitted by the sensor node.

Mean Latency represents the average transmission-to-storage delay of successfully delivered readings and will be computed using Equation (3).

$$\bar{L} = \frac{\sum_{i=1}^n L_i}{n} \quad (3)$$

Where:  $\bar{L}$  = mean end-to-end latency;  $L_i$  = latency of reading  $i$ ;  $n$  = number of successfully delivered readings.

System Throughput determines the rate at which legitimate readings are validated and stored and will be computed using Equation (4).

$$\text{System Throughput} = \frac{N_{\text{processed}}}{T} \quad (4)$$

Where:  $N_{\text{processed}}$  = number of readings successfully processed and stored;  $T$  = test duration.

Transaction Success Rate determines whether submitted hashes and metadata are successfully committed to Fabric and will be computed using Equation (5).

$$\text{TSR}(\%) = \frac{N_{\text{committed}}}{N_{\text{submitted}}} \times 100 \quad (5)$$

Where:  $\text{TSR}(\%)$  = transaction success rate;  $N_{\text{committed}}$  = transactions successfully committed to Hyperledger Fabric;  $N_{\text{submitted}}$  = transactions submitted.

Average CPU Utilization measures the Raspberry Pi processing demand while the integrated services operate and will be computed using Equation (6).

$$\text{CPU}(\%) = \frac{\sum_{i=1}^n \text{CPU}_i}{n} \quad (6)$$

Where:  $\text{CPU}(\%)$  = average CPU utilization;  $\text{CPU}_i$  = CPU utilization at observation  $i$ ;  $n$  = number of observations.

Maximum CPU Utilization identifies the highest Raspberry Pi processor demand during a run and will be computed using Equation (7).

$$\text{CPU}_{\text{max}} = \max(\text{CPU}_1, \text{CPU}_2, \dots, \text{CPU}_n) \quad (7)$$

Where:  $\text{CPU}_{\text{max}}$  = maximum observed CPU utilization;  $\text{CPU}_i$  = CPU utilization at observation  $i$ ;  $n$  = number of observations.

Memory Utilization measures the percentage of the Raspberry Pi's available memory being used by the integrated system at a given observation and will be computed using Equation (8).

$$\text{Memory Utilization}_i(\%) = \frac{\text{Memory}_{\text{used},i}}{\text{Memory}_{\text{total}}} \times 100 \quad (8)$$

Where:  $\text{Memory Utilization}_i(\%)$  = memory utilization at observation  $i$ ;  $\text{Memory}_{\text{used},i}$  = memory used at observation  $i$ ;  $\text{Memory}_{\text{total}}$  = total available memory.

Average Memory Utilization measures the proportion of available Raspberry Pi memory used by the system and will be computed using Equation (9).

$$\text{Mem}\bar{\text{ory}}(\%) = \frac{\sum_{i=1}^n \text{Memory}_i}{n} \quad (9)$$

Where:  $\text{Mem}\bar{\text{ory}}(\%)$  = average memory utilization;  $\text{Memory}_i$  = memory utilization at observation  $i$ ;  $n$  = number of observations.

Maximum Memory Utilization identifies the highest Raspberry Pi memory demand during a run will be computed using Equation (10).

$$\text{Memory}_{\text{max}} = \max(\text{Memory}_1, \text{Memory}_2, \dots, \text{Memory}_n) \quad (10)$$

Where:  $\text{Memory}_{\text{max}}$  = maximum observed memory utilization;  $\text{Memory}_i$  = memory utilization at observation  $i$ ;  $n$  = number of observations

#### 3.2.4.1.2. Hyperledger Fabric Blockchain Performance and Overhead Test

In addition to the end-to-end normal-operation test, the performance of the Hyperledger Fabric layer will be evaluated separately to determine the processing cost introduced by blockchain verification. This assessment is adapted from Shahjalal et al. [39], who evaluated the blockchain layer of a LoRaWAN system using transaction latency, consensus time, transaction size, and block size. In the present study, valid sensor-record hashes and selected metadata will be submitted to Hyperledger Fabric under controlled workloads, and blockchain commit latency, committed transaction throughput, transaction success rate,

transaction size, block size, and Raspberry Pi CPU and memory utilization will be measured. The test will also compare normal processing with and without Fabric submission to estimate the additional performance overhead introduced by blockchain verification. Shahjalal et al. similarly evaluated the processing delay and storage characteristics of the blockchain network layer after LoRaWAN data reached the server.

For the controlled workload, test records will follow the same structure used by the operational prototype. Each record will contain a unique record identifier, timestamp, device identifier, SHA-256 hash, and selected metadata. The records will be submitted through the same chaincode used during normal sensor operation. Four workload conditions will be used: the normal sensor rate of one transaction every 15 seconds, followed by controlled submission rates of 1, 5, and 10 transactions per second. Each workload condition will run for five minutes and will be repeated three times. The blockchain network will be returned to a stable condition before each run, and only valid transactions will be generated so that the experiment measures blockchain performance rather than rejection of invalid traffic.

Table 3.3 summarizes the measures used to evaluate Hyperledger Fabric under the four controlled workload conditions. Commit latency, throughput, and transaction success rate assess the timeliness, capacity, and reliability of blockchain recording, while transaction and block sizes indicate ledger-storage demand. CPU and memory utilization show the processing resources required by Fabric on the Raspberry Pi. Finally, comparing operation with and without Fabric submission determines the additional latency and resource overhead introduced by blockchain verification. Together, these measures show whether the blockchain layer can reliably record sensor hashes and metadata and what performance cost it adds to the prototype.

Table 3.3. Hyperledger Fabric Performance Measures

| System level           | Performance measure         | Rationale                                                                                                         | Performance aspect measured          |
|------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| Transaction processing | Blockchain commit latency   | Determines how long Fabric takes to confirm a submitted sensor hash as a valid ledger transaction.                | Blockchain-recording timeliness      |
| Transaction processing | Mean commit latency         | Shows the typical confirmation delay across all successfully committed transactions.                              | Average blockchain-recording delay   |
| Transaction processing | Transaction throughput      | Determines how many valid sensor-hash transactions Fabric can commit per second as the workload increases.        | Blockchain-processing capacity       |
| Transaction processing | Transaction success rate    | Determines whether submitted sensor hashes and metadata are reliably committed without failure or invalid status. | Blockchain-recording reliability     |
| Blockchain storage     | Average transaction size    | Determines the ledger space required to record each sensor hash and its selected metadata.                        | Per-transaction storage demand       |
| Blockchain storage     | Average block size          | Determines how much ledger storage is consumed by blocks containing the committed transactions.                   | Block-level storage demand           |
| Edge platform          | Average CPU utilization     | Determines the usual Raspberry Pi processing demand while Fabric handles the transaction workload.                | Average blockchain-processing demand |
| Edge platform          | Maximum CPU utilization     | Identifies the highest Raspberry Pi processor demand reached during the workload.                                 | Peak blockchain-processing demand    |
| Edge platform          | Average memory utilization  | Determines the usual Raspberry Pi memory demand while Fabric services and transactions are running.               | Average blockchain-memory demand     |
| Edge platform          | Maximum memory utilization  | Identifies the highest Raspberry Pi memory demand reached during the workload.                                    | Peak blockchain-memory demand        |
| Blockchain integration | Additional latency          | Determines how much processing delay is added when Fabric verification is enabled.                                | Absolute latency overhead            |
| Blockchain integration | Percentage latency increase | Shows the blockchain-related delay relative to processing the same records without Fabric.                        | Relative latency overhead            |
| Blockchain integration | CPU overhead                | Determines the additional Raspberry Pi processing demand caused by enabling Fabric verification.                  | Processing-resource overhead         |
| Blockchain integration | Memory overhead             | Determines the additional Raspberry Pi memory demand caused by enabling Fabric verification.                      | Memory-resource overhead             |

The performance measures presented in Table 3.3 will be calculated using Equations (11)–(24). Blockchain commit latency rate determines how long Fabric takes to confirm a submitted sensor hash as a valid ledger transaction and will be computed using Equation (11).

$$L_{\text{Fabric},i} = T_{\text{commit},i} - T_{\text{submission},i} \quad (11)$$

where:  $L_{\text{Fabric},i}$  = commit latency of transaction  $i$ ;  $T_{\text{commit},i}$  = time transaction is committed to Fabric; and  $T_{\text{submission},i}$  = time transaction is submitted.

Mean commit latency shows the typical confirmation delay across all successfully committed transactions and will be computed using Equation (12).

$$\bar{L}_{\text{Fabric}} = \frac{\sum_{i=1}^n L_{\text{Fabric},i}}{n} \quad (12)$$

where:  $\bar{L}_{\text{Fabric}}$  = mean commit latency;  $L_{\text{Fabric},i}$  = commit latency of transaction  $i$ ; and  $n$  = number of successfully committed transactions.

Transaction throughput determines how many valid sensor-hash transactions Fabric can commit per second as the workload increases and will be computed using Equation (13).

$$\text{Throughput (TPS)} = \frac{N_{\text{committed}}}{T} \quad (13)$$

where:  $N_{\text{Committed}}$  = number of transactions successfully committed;  $T$  = test duration in seconds; and TPS = transactions per second.

Transaction success rate determines whether submitted sensor hashes and metadata are reliably committed without failure or invalid status and will be computed using Equation (14).

$$\text{TSR}(\%) = \frac{N_{\text{committed}}}{N_{\text{submitted}}} \times 100 \quad (14)$$

where: TSR= transaction success rate;  $N_{\text{Committed}}$ = number of successfully committed transactions; and  $N_{\text{submitted}}$ = total number of submitted transactions.

Average transaction size determines the ledger space required to record each sensor hash and its selected metadata and will be computed using Equation (15).

$$\bar{S}_{\text{tx}} = \frac{\sum_{i=1}^n S_{\text{tx},i}}{n} \quad (15)$$

where:  $\bar{S}_{\text{tx}}$ = average transaction size;  $S_{\text{tx},i}$ = size of transaction i; and n= number of measured transactions.

Average block size determines how much ledger storage is consumed by blocks containing the committed transactions and will be computed using Equation (16).

$$\bar{S}_{\text{block}} = \frac{\sum_{j=1}^B S_{\text{block},j}}{B} \quad (16)$$

where:  $\bar{S}_{\text{block}}$ = average block size;  $S_{\text{block},j}$ = size of block ; and B= number of measured blocks.

Average CPU utilization determines the usual Raspberry Pi processing demand while Fabric handles the transaction workload and will be computed using Equation (17).

$$\bar{\text{CPU}}(\%) = \frac{\sum_{i=1}^n \text{CPU}_i}{n} \quad (17)$$

where:  $\bar{\text{CPU}}$ =average CPU utilization;  $\text{CPU}_i$ = CPU utilization recorded at observation i; and n= number of observations.

Maximum CPU utilization identifies the highest Raspberry Pi processor demand reached during the workload and will be computed using Equation (18).

$$\text{CPU}_{\text{max}} = \max(\text{CPU}_1, \dots, \text{CPU}_n)$$

(18)

where:  $CPU_{max}$  = maximum CPU utilization; and  $CPU_1, \dots, CPU_n$  = CPU-utilization observations recorded during the test.

Average memory utilization determines the usual Raspberry Pi memory demand while Fabric services and transactions are running and will be computed using Equation (19).

$$\text{Memory}(\%) = \frac{\sum_{i=1}^n \text{Memory}_i}{n} \quad (19)$$

where:  $\text{Memory}(\%)$  = average memory utilization;  $\text{Memory}_i$  = memory utilization recorded at observation  $i$ ; and  $n$  = number of observations.

Maximum memory utilization identifies the highest Raspberry Pi memory demand reached during the workload and will be computed using Equation (20).

$$\text{Memory}_{max} = \max(\text{Memory}_1, \dots, \text{Memory}_n) \quad (20)$$

where:  $\text{Memory}_{max}$  = maximum memory utilization; and  $\text{Memory}_1, \dots, \text{Memory}_n$  = memory-utilization observations recorded during the test.

Additional latency determines how much processing delay is added when Fabric verification is enabled and will be computed using Equation (21).

$$L_{overhead} = L_{with Fabric} - L_{without Fabric} \quad (21)$$

where:  $L_{overhead}$  = additional latency introduced by blockchain verification;  $L_{with Fabric}$  = latency with Fabric submission; and  $L_{without Fabric}$  = latency without Fabric submission.

Percentage latency increase shows the blockchain-related delay relative to processing the same records without Fabric and will be computed using Equation (22).

$$L_{\text{increase}} (\%) = \frac{L_{\text{with Fabric}} - L_{\text{without Fabric}}}{L_{\text{without Fabric}}} \times 100 \quad (22)$$

where:  $L_{\text{increase}} (\%)$  = percentage increase in latency;  $L_{\text{with Fabric}}$  = latency with Fabric submission; and  $L_{\text{without Fabric}}$  = latency without Fabric submission. .

CPU overhead determines the additional Raspberry Pi processing demand caused by enabling Fabric verification and will be computed using Equation (23).

$$\text{CPU}_{\text{overhead}} = \text{CPU}_{\text{with Fabric}} - \text{CPU}_{\text{without Fabric}} \quad (23)$$

where:  $\text{CPU}_{\text{overhead}}$  = additional CPU utilization introduced by Fabric;  $\text{CPU}_{\text{with Fabric}}$  = CPU utilization with Fabric; and  $\text{CPU}_{\text{without Fabric}}$  = CPU utilization without Fabric.

Memory overhead determines the additional Raspberry Pi memory demand caused by enabling Fabric verification and will be computed using Equation (24).

$$\text{Memory}_{\text{overhead}} = \text{Memory}_{\text{with Fabric}} - \text{Memory}_{\text{without Fabric}} \quad (24)$$

where:  $\text{Memory}_{\text{Overhead}}$  = additional memory utilization introduced by Fabric;  $\text{Memory}_{\text{with Fabric}}$  = memory utilization with Fabric; and  $\text{Memory}_{\text{without Fabric}}$  = memory utilization without Fabric. .

### 3.2.4.2. Security and Resilience Assessment

The security of the prototype will be evaluated through a multilayer security assessment covering the device and LoRaWAN communication layer, edge and network layer, application and access-control layer, storage layer, and blockchain layer. This approach recognizes that threats may occur at different points in the data flow, from sensor transmission and gateway processing to database storage and blockchain verification.

Table 3.4 presents the tests proposed to assess the security and resilience of the developed architecture. Based on the agricultural literature reviewed during

the Analysis and Needs Assessment phase, six relevant areas were identified: authentication and access control, replay and spoofing resistance, DoS or flooding resistance, data integrity, traceability and accountability, and resilience and recovery. These areas were mapped to relevant NIST SP 800-53 controls to confirm their alignment with established security and resilience guidance. The table then organizes the proposed tests under the study's four objective areas and identifies the procedure and system layer involved in each assessment.

Table 3.4. Security and Resilience Assessment Plan

| Study security and resilience objective area | Synthesized security and resilience area in smart agriculture        | Proposed test                                                | Procedure                                                                                                           | Main layer                           |
|----------------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| Communication and service failures           | Resilience and Recovery [72], [84], [79], [87]                       | Internet-connectivity interruption                           | Interrupt Internet connectivity while the Raspberry Pi and local services continue operating                        | Edge and network                     |
|                                              |                                                                      | Internet-connectivity recovery                               | Restore Internet connectivity and check for missing or duplicate records                                            | Edge and network                     |
|                                              |                                                                      | Database–blockchain consistency during Fabric unavailability | Make Fabric unavailable and verify locally stored records remain PENDING, not VERIFIED                              | Application and storage              |
|                                              |                                                                      | Database–blockchain recovery and reconciliation              | Restore Fabric and reconcile pending hashes, checking commitment, duplicates, missing records, and hash consistency | Application, storage, and blockchain |
| Multilayer security                          | Authentication and access control [72], [73], [80], [78], [87], [88] | LoRaWAN device authentication                                | Use an unregistered LoRaWAN device to transmit sensor data                                                          | Device and LoRaWAN                   |
|                                              |                                                                      | MQTT authentication and authorization                        | Attempt access or publishing using invalid or unauthorized credentials                                              | Application / MQTT                   |
|                                              |                                                                      | Hyperledger Fabric authorization                             | Unauthorized Fabric identity attempts to submit a sensor-hash transaction                                           | Blockchain                           |
|                                              |                                                                      | Hyperledger Fabric endorsement-policy enforcement            | Attempt to create a blockchain integrity record without all required endorsements                                   | Blockchain                           |

|                |                                                                  |                                      |                                                                                                |                        |
|----------------|------------------------------------------------------------------|--------------------------------------|------------------------------------------------------------------------------------------------|------------------------|
|                | Replay or spoofing<br>[77], [81], [83]                           | LoRaWAN replay                       | Retransmit a previously captured LoRaWAN packet                                                | LoRaWAN                |
|                |                                                                  | Application-layer duplicate/replay   | Resubmit a previously processed or duplicate sensor record                                     | Application            |
|                | DoS or flooding<br>[74], [75], [76], [77], [81], [82], [87]      | MQTT connection flooding             | Send repeated invalid MQTT connection attempts while legitimate traffic continues              | Edge / MQTT            |
|                |                                                                  | Invalid application-message flooding | Send increasing invalid application messages while legitimate records continue to be processed | Application            |
| Data integrity | Data integrity<br>[72], [73], [74], [77], [84], [87], [91], [92] | Application-layer integrity          | Alter a sensor value during Node-RED processing after the initial hash is generated            | Application            |
|                |                                                                  | Post-storage integrity               | Alter a PostgreSQL/TimescaleDB value after its original hash is committed to Fabric            | Storage                |
|                |                                                                  | Blockchain-layer integrity           | Attempt to duplicate or replace an existing sensor-hash anchor using the same trace_id         | Blockchain             |
| Traceability   | Traceability and accountability [73], [82], [89], [90]           | Individual sensor-record trace       | Retrieve the sensor record, device ID, timestamp, hash, and corresponding Fabric transaction   | Storage and blockchain |
|                |                                                                  | Device-history reconstruction        | Retrieve and reconstruct the chronological history of records for a selected device            | Storage and blockchain |

Figure 3.6 shows how security and resilience are assessed as agricultural sensor data move through the prototype—from the LoRaWAN gateway and Node-RED to PostgreSQL/TimescaleDB and Hyperledger Fabric. The assessment covers the six areas identified from the agricultural literature. Authentication and access-control tests determine whether unauthorized devices, MQTT users, or Fabric identities can enter the system or create blockchain records. Replay and spoofing tests examine how the prototype handles retransmitted packets and duplicate records, while integrity tests introduce changes during processing, after

storage, and at the blockchain layer. Traceability is examined by following an individual reading and reconstructing a device's record history. Flooding tests assess whether legitimate data can still be processed during invalid MQTT connections or application messages. Finally, resilience tests interrupt Internet and Fabric services to determine whether local operation continues and whether stored records are correctly reconciled after service is restored.

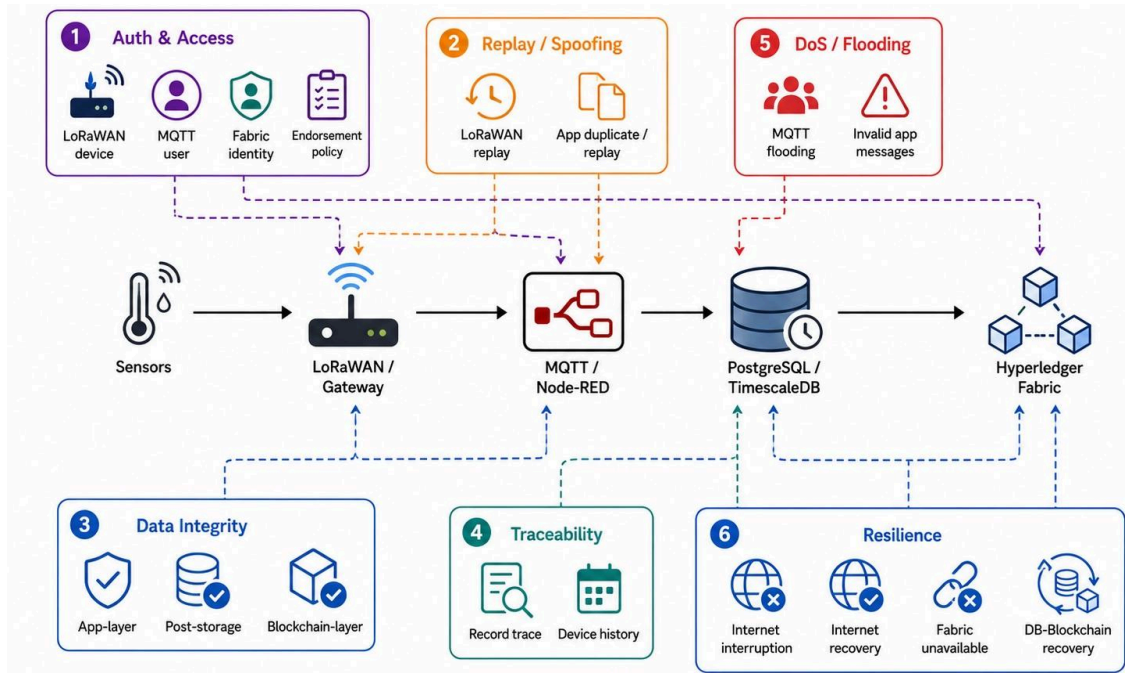


Figure 3.6. Security and Resilience Test Scheme for the Prototype

### 3.2.4.2.1 Communication and Service Failures

This assessment examines whether the prototype can continue essential monitoring during temporary Internet or blockchain-service interruptions and recover without losing, duplicating, or incorrectly verifying agricultural records.

The Analysis and Needs Assessment phase showed that agricultural IoT systems must preserve essential operations and support response, backup, and recovery following attacks, failures, or service interruptions [72], [84], [87]. They must also account for threats affecting agricultural processes, communication, services, hardware, software, and data [79]. However, practical resilience testing

in agricultural IoT remains limited. Hou et al. [40] addressed resilience by reducing single points of failure but did not directly interrupt a gateway or server, whereas Homaei et al. [47] interrupted Internet connectivity and examined whether the gateway retained and later submitted pending readings without data loss or blockchain conflict. Drawing from these concerns and prior tests, the present study interrupts Internet connectivity to determine whether local monitoring continues, then restores the connection to check for missing or duplicate records. Fabric is also made temporarily unavailable to verify that locally stored records remain **PENDING**, rather than being incorrectly marked **VERIFIED**, and is subsequently restored to examine whether pending hashes are committed and reconciled without duplication, loss, or inconsistency. These procedures also reflect NIST SP 800-53 CP-10 and SC-24, which call for recovery after disruption and return to a known system state. Because NIST specifies the expected control outcome rather than an exact experiment, the tests are adapted to the services and data flow of the proposed architecture—a common approach when translating general security controls into system-specific evaluation procedures.

#### **A. Internet-Connectivity Interruption and Recovery Test**

The study will conduct an Internet-connectivity interruption and recovery test to determine whether the prototype can continue collecting, processing, storing, displaying, and verifying agricultural sensor readings without external Internet access. Only the Raspberry Pi's wide-area Internet connection will be disabled; the sensor, LoRaWAN gateway, Raspberry Pi, local network, and locally hosted services will remain powered and operational.

Each two-hour run will consist of 30 minutes of normal operation, 60 minutes without Internet connectivity, and 30 minutes after reconnection. The sensor will transmit one reading every 15 seconds, producing approximately 120,

240, and 120 readings during the three periods, respectively. The complete procedure will be repeated three times under the same configuration.

Table 3.5 summarizes the three consecutive conditions used in each experimental run. The normal-operation period establishes the baseline for comparison, while the interruption period determines whether locally hosted services can continue receiving, processing, storing, displaying, and verifying readings without external Internet access. The recovery period then examines whether connectivity returns without disrupting local services or causing additional missing, duplicate, disordered, or inconsistent database and blockchain records. Evidence collected during these periods allows changes in service availability, data continuity, latency, and database–blockchain consistency to be compared.

Table 3.5. Internet-Connectivity Interruption and Recovery Test Design

| Test condition                       | Duration   | Expected readings | Expected system behavior                                                                                           | Evidence collected                                                           |
|--------------------------------------|------------|-------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Normal operation before interruption | 30 minutes | Approximately 120 | Readings are received, processed, stored, displayed, and verified normally                                         | Service logs, database records, and Fabric transactions                      |
| Internet interruption                | 60 minutes | Approximately 240 | Local collection, processing, storage, visualization, and blockchain verification continue without Internet access | Local service logs, database records, Fabric transactions, and resource logs |
| Recovery after reconnection          | 30 minutes | Approximately 120 | Internet access returns without service failure, missing records, duplicate records, or incorrect record order     | Connectivity logs, timestamps, database records, and Fabric transactions     |

### Test Procedure

Before each run, the researcher will confirm that ChirpStack, Mosquitto MQTT, Node-RED, PostgreSQL/TimescaleDB, Grafana, and Hyperledger Fabric are operating normally. Sensor readings will then be transmitted for 30 minutes to

establish the baseline condition, and the transmitted, received, stored, and blockchain-recorded readings will be documented.

The Raspberry Pi's external Internet connection will then be disabled for 60 minutes while the local network remains active. During this period, system logs and records will be examined to determine whether the locally hosted services continue receiving, processing, storing, displaying, and verifying sensor readings.

Internet connectivity will then be restored, and the prototype will be observed for another 30 minutes. Sensor-transmission logs, frame counters, timestamps, trace identifiers, database records, and Fabric transactions will be compared across the three periods to identify missing, duplicated, incorrectly ordered, or inconsistent records. Figure 3.7 illustrates the three periods of the Internet-connectivity interruption and recovery test: 30 minutes of normal operation, 60 minutes without Internet access, and 30 minutes after reconnection. It shows that local services will remain operational during the interruption and that logs, timestamps, database records, and Fabric transactions will be examined after recovery for missing, duplicate, disordered, or inconsistent records.

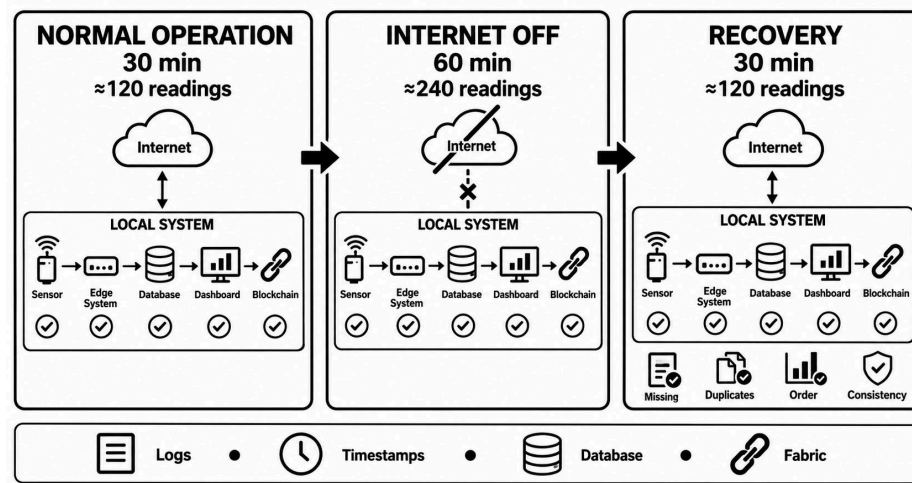


Figure 3.7. Internet-Connectivity Interruption and Recovery Test Flow

## Measures and Data Analysis

The test will examine:

- **Local service availability:** whether each locally hosted service remains operational during the interruption.
- **Data-delivery rate:** the percentage of transmitted readings successfully received and stored.
- **Data-loss count:** the number of transmitted readings not found in the database.
- **Duplicate-record count:** the number of readings stored more than once.
- **Database–blockchain consistency:** the percentage of stored records correctly linked to their Fabric hashes and transactions.
- **End-to-end latency:** the time from sensor transmission to database storage during each test period.
- **Recovery time:** the time from Internet restoration until external connectivity returns to its normal condition.
- **Chronological accuracy:** whether readings remain correctly ordered according to their timestamps and frame counters.

Frequencies and percentages will summarize service availability, delivery, missing records, duplicates, chronological accuracy, and database–blockchain consistency. End-to-end latency and recovery time will be summarized using the mean and standard deviation, and the results will be reported separately for the normal, interruption, and recovery periods. The test is

The prototype will demonstrate resilience if its essential local monitoring functions continue during the interruption and it returns to normal connectivity without database–blockchain inconsistency. Because ordinary LoRaWAN transmission loss may occur independently of the Internet outage, the test will determine whether the interruption causes **additional** data loss, duplication, or incorrect record linkage relative to the baseline. The test does not evaluate Raspberry Pi power failure or complete gateway failure.

## **B. Database–Blockchain Consistency and Recovery Test**

The database–blockchain consistency and recovery test will determine whether the prototype maintains consistency between sensor records stored locally in PostgreSQL/TimescaleDB and their corresponding integrity hashes recorded in Hyperledger Fabric when the blockchain service becomes temporarily unavailable. The test is informed by the hybrid storage approach of Honar Pajooch et al. [43], in which IoT data are maintained outside the blockchain while checksums are recorded in Hyperledger Fabric for subsequent integrity verification. It is also informed by Zhang et al. [64], who showed that inconsistencies can arise when an off-chain application updates its state before the corresponding blockchain transaction has reached its confirmed state. In their study, premature updating of off-chain state while a blockchain transaction remained pending was identified as a synchronization problem. Na et al. [65] similarly used blockchain-recorded metadata to verify whether stored data remained synchronized and initiated a recovery process when inconsistency was detected. Accordingly, the present study will extend these principles to the PostgreSQL/TimescaleDB–Hyperledger Fabric architecture by distinguishing records awaiting blockchain confirmation from records whose hashes have been successfully committed and by testing whether pending records can be correctly reconciled after Fabric service restoration.

Each valid sensor reading will be assigned a unique `trace_id` by Node-RED before storage. The complete canonical sensor record will be stored in PostgreSQL/TimescaleDB, while its `trace_id`, SHA-256 hash, device identifier, timestamp, and selected metadata will be submitted to Hyperledger Fabric. A `blockchain-status` field will be maintained in PostgreSQL/TimescaleDB to represent the state of the blockchain operation. A record will initially be assigned the status `PENDING` after successful local storage. It will be changed to `VERIFIED` only after successful Fabric commitment has been confirmed and the corresponding transaction identifier has been recorded. A `FAILED` status will be assigned only when a record cannot be successfully reconciled after the defined

retry procedure and therefore requires further investigation. This prevents a locally stored record from being represented as blockchain-verified before a corresponding valid blockchain record can be confirmed. The distinction between pending and confirmed states is consistent with the synchronization principle described by Zhang et al. [64], who emphasized that off-chain state should not be treated as finalized while the associated blockchain operation remains unresolved.

### **Normal-Operation Condition**

For the control condition, Hyperledger Fabric and all other prototype services will operate normally. Ten valid sensor readings will be transmitted through the normal data flow from the LoRaWAN sensor to ChirpStack, MQTT, Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric. After each reading is stored in PostgreSQL/TimescaleDB, its blockchain status will initially be set to PENDING. Node-RED will then submit the corresponding SHA-256 hash and metadata to Fabric.

After successful blockchain commitment is confirmed, the Fabric transaction identifier will be written to the corresponding PostgreSQL/TimescaleDB record and the blockchain status will be changed from PENDING to VERIFIED. The database record, recorded hash, transaction identifier, and Fabric ledger entry will then be compared using the common trace\_id. All ten control records should reach the VERIFIED state with matching database and blockchain hashes.

### **Fabric-Unavailability Condition**

For the fault condition, Hyperledger Fabric will be deliberately made unavailable while the sensor, Raspberry Pi gateway, ChirpStack, MQTT, Node-RED, and PostgreSQL/TimescaleDB remain operational. The interruption will be introduced only after normal system operation has first been confirmed.

Ten additional sensor readings will then be transmitted. Node-RED should continue receiving and processing the sensor data, generating the canonical

records and SHA-256 hashes, and storing the complete records in PostgreSQL/TimescaleDB. Because Fabric commitment cannot be confirmed during this condition, the records must remain marked as PENDING. They must not be marked as VERIFIED, and no Fabric transaction identifier will be fabricated or assigned to them.

For each of the ten records, the researcher will verify that the complete sensor record remains available in PostgreSQL/TimescaleDB, that its SHA-256 hash has been retained for later submission, and that its blockchain status correctly indicates that blockchain verification is still pending. Node-RED and Fabric communication logs will also be retained as evidence of the unsuccessful blockchain-submission attempts.

### **Blockchain Restoration and Reconciliation Condition**

After all ten fault-condition records have been stored, Hyperledger Fabric will be restored. A reconciliation function in Node-RED will identify all records whose blockchain status remains PENDING. The same ten records generated during the Fabric-unavailability condition will be used for this recovery stage; no additional test records will be created.

Before resubmitting a pending hash, the reconciliation function will query Fabric using the record's unique `trace_id`. This check is intended to prevent unnecessary duplicate submissions in cases where a transaction may have been committed but its confirmation was not successfully returned to the application. Three possible results will be handled.

If no blockchain record exists for the `trace_id`, Node-RED will resubmit the stored SHA-256 hash and metadata to Fabric. Once commitment is confirmed, the Fabric transaction identifier will be stored and the database status will change from PENDING to VERIFIED.

If the `trace_id` already exists in Fabric and its blockchain hash matches the locally stored hash, the existing commitment will be recognized and the

corresponding database record will be updated to **VERIFIED** without creating another integrity anchor.

If the same `trace_id` exists but contains a different hash, the record will not be automatically verified. It will be marked **FAILED** or placed in the test exception log for investigation because the database and blockchain representations are inconsistent.

After reconciliation, all recovered records will be checked again. The canonical database record will be reconstructed and its SHA-256 hash recomputed. The recomputed hash will then be compared with the hash associated with the same `trace_id` in Hyperledger Fabric. The transaction history will also be examined to determine whether any expected blockchain records are missing or whether more than one valid integrity anchor has been created for the same record.

This recovery mechanism is consistent with the principle used by Na et al. [65], whose system compares stored-data hashes with blockchain information and initiates recovery when the data are found to be out of synchronization. Their architecture also explicitly records synchronization and recovery metadata on the blockchain. The specific PostgreSQL/TimescaleDB-to-Fabric retry procedure used in the present study, however, is an adaptation developed for the proposed prototype. Figure 3.8 illustrates the five stages of the Hyperledger Fabric unavailability and recovery test, from normal verification to retaining records as **PENDING** while Fabric is unavailable. After service restoration, the pending hashes will be submitted and reconciled, with records marked **VERIFIED** only after confirming that no loss, duplication, false verification, or hash inconsistency occurred.

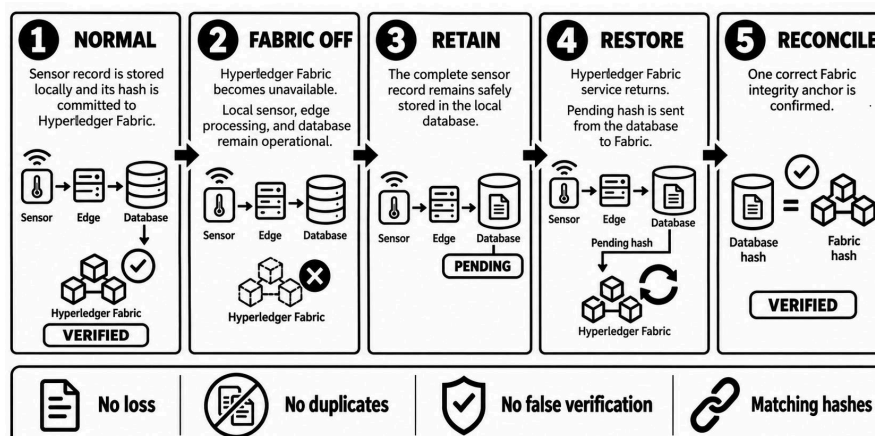


Figure 3.8. Hyperledger Fabric Unavailability and Recovery Test

### Measures and Data Analysis

For every record, the researcher will record the trace\_id, database-storage result, initial blockchain status, Fabric-submission result, final blockchain status, Fabric transaction identifier where applicable, original SHA-256 hash, recomputed database hash, blockchain hash, and relevant timestamps. The following measures will be used.

The **pending-record identification rate** in equation (25) will measure whether records generated while Fabric is unavailable are correctly retained as awaiting blockchain confirmation:

$$\text{Pending-Record Identification Rate} = \frac{N_{\text{outage records correctly marked PENDING}}}{N_{\text{records generated during Fabric unavailability}}} \times 100 \quad (25)$$

where:  $N_{\text{outage records correctly marked PENDING}}$  number of records generated during Fabric unavailability that are correctly marked **PENDING**; and  $N_{\text{records generated during Fabric unavailability}}$  total number of records generated while Fabric is unavailable.

The **recovery success rate** in equation (26) will measure the proportion of pending records that are successfully reconciled after Fabric is restored:

$$\text{Recovery Success Rate} = \frac{N_{\text{pending records successfully VERIFIED after recovery}}}{N_{\text{pending records requiring reconciliation}}} \times 100 \quad (26)$$

where:  $N_{\text{pending records successfully VERIFIED after recovery}}$ : number of pending records successfully reconciled and marked **VERIFIED** after Fabric is restored; and  $N_{\text{pending records requiring reconciliation}}$ : total number of pending records requiring reconciliation.

The **hash-consistency rate after recovery** in equation (27) will measure whether the recovered database records correspond to their Fabric integrity anchors:

$$\text{Hash-Consistency Rate} = \frac{N_{\text{recovered records with matching database and Fabric hashes}}}{N_{\text{recovered records verified}}} \times 100 \quad (27)$$

where:  $N_{\text{recovered records with matching database and Fabric hashes}}$  = number of recovered records whose database and Fabric hashes match; and  $N_{\text{recovered records verified}}$  = total number of recovered records subjected to hash verification.

The number of **false verifications** will also be recorded. A false verification occurs when a record is marked VERIFIED even though no corresponding valid Fabric commitment can be confirmed. In addition, the researcher will record the number of orphaned database records, missing blockchain records, duplicate blockchain records, conflicting hashes, failed reconciliation attempts, and records remaining PENDING after the recovery procedure.

**Recovery time** will be measured from the time Fabric becomes available again until all recoverable PENDING records have either reached VERIFIED status or have been classified as FAILED. The mean per-record reconciliation time may also be reported where appropriate.

Frequencies and percentages will be used to summarize the consistency and recovery outcomes, while recovery time will be reported in seconds. The security and resilience requirements will be a 100% pending-record identification rate, zero false verifications, 100% recovery success for valid pending records, 100% post-recovery hash consistency, and zero missing or duplicate valid blockchain integrity anchors. The expected outcome is that records generated while Fabric is unavailable remain safely stored and identifiable as PENDING, and that after Fabric is restored, each pending record is reconciled to one correct blockchain integrity anchor before its status is changed to VERIFIED.

This test therefore evaluates not merely whether Hyperledger Fabric can restart, but whether the hybrid database–blockchain state remains trustworthy across a temporary blockchain-service interruption, which is the central resilience issue created by separating complete sensor storage from blockchain-based integrity verification.

Table 3.6 summarizes how database and blockchain states will be examined before, during, and after Fabric unavailability. Normal operation establishes whether records move correctly from PENDING to VERIFIED. During Fabric unavailability, the test determines whether readings remain locally available without being falsely represented as blockchain-verified. After Fabric is restored, the same pending records are reconciled by checking their trace\_id and hash before any status is changed. Together, the three conditions determine whether the prototype preserves trustworthy database–blockchain state across a temporary blockchain-service interruption.

Table 3.6. Database–Blockchain Consistency and Recovery Test Design

| Test condition                        | Records used             | Expected database status                                           | System action                                                                                                         | Expected evidence                                                                                                       |
|---------------------------------------|--------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Normal operation                      | 10 valid readings        | PENDING followed by VERIFIED                                       | Store each complete record, submit its hash and metadata to Fabric, and update its status after confirmed commitment  | Matching trace_id, database hash, Fabric hash, and transaction ID                                                       |
| Fabric unavailability                 | 10 valid readings        | Remains PENDING                                                    | Continue local processing and storage while retaining hashes and metadata for later submission                        | Complete database records, retained hashes, no fabricated transaction IDs, and zero false verifications                 |
| Fabric restoration and reconciliation | Same 10 pending readings | VERIFIED after successful reconciliation or FAILED if inconsistent | Query Fabric by trace_id, submit missing hashes, recognize matching existing commitments, and flag conflicting hashes | One valid integrity anchor per record, matching hashes, no missing or duplicate commitments, and recorded recovery time |

Together, these tests address the communication and service-failure objective by examining whether the prototype can maintain essential agricultural monitoring and recover correctly from two forms of disruption. The Internet-connectivity tests determine whether local collection, processing, storage, visualization, and blockchain verification continue without external connectivity and whether reconnection occurs without missing or duplicate records. The Fabric-unavailability test determines whether locally stored records remain

correctly marked as PENDING when blockchain verification cannot be confirmed, while the recovery and reconciliation test examines whether those records are subsequently linked to one valid blockchain transaction without missing records, duplicate commitments, or hash inconsistencies. Collectively, the results show whether the architecture can continue operating and restore a consistent system state after communication or blockchain-service failure.

#### 3.2.4.2.2 Multilayer Security

The second assessment area examines security across the different layers through which agricultural data pass. The Analysis and Needs Assessment phase established the agricultural relevance of these threats, although practical security testing in agricultural systems remains limited. The study therefore draws on testing approaches used in related IoT and blockchain systems and adapts them to the components and data flow of the proposed architecture.

As summarized in Table 3.4, **(A) authentication and access control** determine whether unauthorized LoRaWAN devices, MQTT users, or Fabric identities can access the system or create blockchain records; **(B) replay and spoofing resistance** determines how the LoRaWAN and application layers handle retransmitted packets and duplicate sensor records; and **(C) DoS or flooding resistance** assesses whether legitimate monitoring data can still be processed while the MQTT and application layers receive increasing invalid traffic. Each area and its corresponding tests are discussed in the following subsections.

##### A. Authentication and Access Control

The authentication and access-control test will determine whether the prototype accepts only registered devices and authorized system users while rejecting invalid credentials and prohibited actions. The proposed method combines the invalid-device test of Hou et al. [40], the allow-or-deny test of Iftekhhar et al. [41], and the unauthorized

blockchain-access test of Homaei et al. [47]. Unlike these earlier studies, the proposed test will examine authentication and authorization at four points in the data flow: LoRaWAN device activation, MQTT messaging, PostgreSQL/TimescaleDB storage, and Hyperledger Fabric transactions.

The test will determine whether the prototype accepts registered devices and authorized identities while rejecting invalid credentials and prohibited actions. Testing will be conducted on an isolated network using synthetic sensor data. Logs from ChirpStack, Mosquitto MQTT, Node-RED, and Hyperledger Fabric will be enabled before the test. Each condition will be repeated 10 times, giving a total of 90 test attempts. Ten repetitions are proposed because the test evaluates the consistency of deterministic allow-or-deny controls rather than estimating behavior in a large population.

Table 3.7. Authentication and Access-Control Test Design

| Test area                             | Test condition                                   | Number of trials | Measure                                           | Expected result                              | Evidence collected                                    |
|---------------------------------------|--------------------------------------------------|------------------|---------------------------------------------------|----------------------------------------------|-------------------------------------------------------|
| LoRaWAN device authentication         | Registered device with correct OTAA credentials  | 10               | Join decision and response time                   | Device joins and transmits data              | ChirpStack join event, JoinAccept and accepted uplink |
|                                       | Registered DevEUI with incorrect AppKey          | 10               | Rejection decision and unauthorized data received | Join request is rejected                     | ChirpStack event and absence of JoinAccept            |
|                                       | Unregistered DevEUI                              | 10               | Rejection decision and unauthorized data received | Device is not activated                      | ChirpStack event and absence of accepted data         |
| MQTT authentication and authorization | Authorized account publishes to an allowed topic | 10               | Publish decision and response time                | Message is accepted and received by Node-RED | Mosquitto and Node-RED logs                           |
|                                       | Account uses an incorrect password               | 10               | Connection decision and response time             | Connection is rejected                       | Mosquitto authentication error                        |
|                                       | Limited account                                  | 10               | Publish                                           | Connection                                   | Mosquitto                                             |

|                                  |                                                         |    |                                            |                                       |                                                       |
|----------------------------------|---------------------------------------------------------|----|--------------------------------------------|---------------------------------------|-------------------------------------------------------|
|                                  | publishes to a prohibited topic                         |    | decision and unauthorized message received | may succeed, but publishing is denied | access-control log and absence of message in Node-RED |
| Hyperledger Fabric authorization | Authorized writer submits a sensor hash                 | 10 | Transaction decision and response time     | Transaction is endorsed and committed | Transaction ID, chaincode and peer logs               |
|                                  | Valid identity without writer permission submits a hash | 10 | Rejection decision and state change        | Transaction is rejected               | Authorization error and unchanged world state         |
|                                  | Untrusted or invalid identity submits a transaction     | 10 | Identity decision and state change         | Connection or transaction is rejected | Fabric identity or certificate error                  |

### Test 1. LoRaWAN Device Authentication Procedure

Three sensor configurations will be prepared: one registered device with the correct DevEUI, JoinEUI and AppKey; one device using a registered DevEUI but an incorrect AppKey; and one device using an unregistered DevEUI. For each trial, the sensor will be restarted and instructed to perform an OTAA join. ChirpStack will be observed to determine whether the JoinRequest is received and whether a JoinAccept is generated. When the authorized device joins successfully, it will send one test reading to confirm that the data reaches MQTT. Each of the three conditions will be repeated 10 times. A 30-second interval will be provided between attempts to avoid overlapping join procedures.

### Test 2. Application/MQTT Access Control procedure

Two MQTT accounts will be configured. The authorized Node-RED account will be permitted to access the required sensor topics, while a limited test account will be prohibited from publishing to protected topics. Anonymous access will be disabled. From a separate test laptop, the researcher will attempt an authorized publish, a connection using an incorrect password, and a publish using valid credentials but a

prohibited topic. Each condition will be repeated 10 times using `mosquitto_pub`. Mosquitto and Node-RED logs will be checked to confirm that denied messages did not enter the processing pipeline.

### **Test 3. Hyperledger Fabric Identity Access Control procedure**

Three Fabric identities will be prepared: an authorized writer, a valid user without writing permission, and an invalid or untrusted identity. The authorized writer will submit a synthetic sensor hash and metadata through the chaincode. The other two identities will attempt to invoke the same function. Each condition will be repeated 10 times. Before and after every unauthorized attempt, the corresponding blockchain record will be queried to confirm that the world state did not change.

### **Measures and Data Analysis**

For every trial, the researcher will record the expected decision, actual decision, response time, system message or error, and whether the request caused any unauthorized message or blockchain state change. The assessment will use the following measures:

- **Authorized success rate:** percentage of valid requests correctly accepted.
- **Unauthorized rejection rate:** percentage of invalid or prohibited requests correctly rejected.
- **False acceptance:** an unauthorized request that is incorrectly accepted.
- **False rejection:** an authorized request that is incorrectly rejected.
- **Correct-decision rate:** percentage of all requests correctly classified.
- **Response time:** time required for the system to accept, reject, or commit the request.
- **Unauthorized state change:** whether a prohibited request created or modified a protected record.

Response time will be measured separately at each layer. For LoRaWAN, it will be measured from the recorded JoinRequest to the

JoinAccept or final rejection decision. For MQTT, it will be measured from the connection or publish request to the broker response. For Hyperledger Fabric, it will be measured from transaction submission to commitment or rejection.

The results will be reported separately for the three layers because each uses a different authentication mechanism. Frequencies and percentages will be used for allow-or-deny outcomes, while the mean and standard deviation will summarize response time. The main security requirement will be zero false acceptance and zero unauthorized state change. Authorized communication failures caused by packet loss will be identified through ChirpStack and gateway logs and will not automatically be classified as authentication failures.

#### **Test 4. Hyperledger Fabric Endorsement-Policy Enforcement Procedure**

The Hyperledger Fabric endorsement-policy enforcement test will determine whether a sensor-hash transaction can create a valid blockchain state change when the endorsements required by the configured policy are not obtained. The procedure is adapted from Iftekhhar et al. [41], who experimentally evaluated authorization and policy enforcement in a Hyperledger Fabric IoT system and demonstrated that transactions were permitted or denied according to the configured access and endorsement requirements. The test is further supported by Soelman et al. [63], who explained that Fabric endorsement policies specify which organizations or peers must confirm a transaction before it can be considered valid and admitted to the ledger. They further showed that requiring only one endorsement may allow one organization to make ledger changes unilaterally, whereas requiring endorsements from multiple independent organizations can strengthen ledger integrity.

For the present prototype, the Hyperledger Fabric network will include two organizations for the test: a Farm/IoT organization (FarmOrg) representing the operational IoT system and an Auditor/IT organization (AuditOrg) representing an independent verifying participant. Each organization will have an endorsing peer. The endorsement policy for creating a sensor-hash integrity record will require endorsements from both organizations before the transaction can produce a valid state change. This arrangement follows the multi-party authorization principle described by Soelman et al. [63], who proposed the inclusion of independent or neutral organizations in endorsement policies to reduce the possibility that one participant can unilaterally introduce an invalid ledger change.

Three conditions will be evaluated. For the normal endorsement condition, 10 valid sensor-hash transactions will be submitted while the required endorsers from both FarmOrg and AuditOrg are available. Each transaction will contain a unique `trace_id`, SHA-256 hash, device identifier, and timestamp. Both required organizations should endorse the transaction, after which it should be ordered, validated, and committed to the Fabric ledger. The transaction identifier and resulting world-state entry will be recorded as evidence of successful authorization.

For the policy-violation condition, 10 new sensor-hash transactions will be submitted without obtaining the endorsement of AuditOrg while the configured policy continues to require endorsements from both FarmOrg and AuditOrg. This condition represents an otherwise authorized FarmOrg application attempting to create a blockchain integrity record without the required independent endorsement. The transaction should not produce a valid committed state change because the endorsement requirement has not been satisfied. Fabric client, gateway, peer, and chaincode logs will be retained to identify the policy-related rejection, while the world state will be queried before and after each attempt to confirm that no new valid sensor-hash record was created.

The test focuses on the absence of the required endorsement, rather than simply on peer failure. Temporarily making the AuditOrg endorser unavailable may be used to produce this condition, but the security property being evaluated is whether the configured endorsement policy prevents a transaction from becoming valid when one required organizational endorsement is missing. This distinction is important because Soelman et al. [63] noted that requiring particular organizations in an endorsement policy also creates an availability dependency when those endorsers become unavailable.

For the recovery condition, the AuditOrg endorser will be restored and 10 additional valid sensor-hash transactions will be submitted using the same endorsement policy. The transactions should again obtain the required FarmOrg and AuditOrg endorsements and be successfully committed. This condition will determine whether normal blockchain operation resumes after the required endorsement becomes available again. The recovery condition extends the policy-enforcement test by examining whether the prototype returns to its expected authorization state after the temporary test condition is removed.

### Measures and Data Analysis

For every transaction attempt, the researcher will record the test condition, trace\_id, endorsement result, transaction identifier where applicable, commit status, validation result, and whether a valid world-state change occurred. The principal measure will be the **endorsement-policy enforcement rate** in equation (28), representing the percentage of transactions lacking a required endorsement that are correctly prevented from producing a valid ledger state change:

$$\text{Endorsement-Policy Enforcement Rate} = \frac{N_{\text{policy-violating attempts blocked}}}{N_{\text{policy-violating attempts}}} \times 100 \quad (28)$$

where:  $N_{\text{policy-violating attempts blocked}}$  = number of policy-violating transaction attempts correctly blocked; and  $N_{\text{policy-violating attempts}}$  = total number of transactions submitted without the required endorsement.

The **valid transaction success rate** in equation (29) will measure the percentage of legitimate transactions satisfying the required endorsement policy that are successfully committed:

$$\text{Valid Transaction Success Rate} = \frac{N_{\text{valid transactions committed}}}{N_{\text{valid transactions submitted}}} \times 100 \quad (29)$$

where:  $N_{\text{valid transactions committed}}$  = number of legitimate transactions satisfying the endorsement policy that are successfully committed; and  $N_{\text{valid transactions submitted}}$  = total number of legitimate transactions submitted with the required endorsements.

For the recovery condition, the **recovery transaction success rate** measures the percentage of valid transactions successfully committed after the Hyperledger Fabric service is restored and it will be calculated based on equation (30):

$$\text{Recovery Transaction Success Rate} = \frac{N_{\text{post-restoration transactions committed}}}{N_{\text{post-restoration transactions submitted}}} \times 100 \quad (30)$$

where:  $N_{\text{post-restoration transactions committed}}$  = number of valid transactions successfully committed after Fabric service is restored; and  $N_{\text{post-restoration transactions submitted}}$  = total number of valid transactions submitted after service restoration.

The unauthorized state-change count will also be recorded. An unauthorized state change will occur if a transaction that does not satisfy the required FarmOrg–AuditOrg endorsement policy nevertheless creates or modifies a valid sensor-hash record in the Fabric world state.

Frequencies and percentages will be used to summarize the endorsement and commit decisions for the three conditions. The security requirement will be a 100% endorsement-policy enforcement rate and zero unauthorized state changes during the policy-violation condition. All legitimate transactions should be successfully committed when the required endorsements are available, no transaction lacking the required AuditOrg endorsement should produce a valid blockchain state change,

and normal transaction processing should resume after the required endorser is restored.

This test therefore evaluates a different aspect of blockchain authorization from the existing Fabric-identity test. The identity test determines whether an unauthorized Fabric user can submit transactions, whereas the endorsement-policy test determines whether an authorized participant can act unilaterally when the blockchain policy requires approval from another organization. This directly demonstrates the multi-party authorization function of the permissioned blockchain rather than merely assuming that the configured endorsement policy operates as intended.

## **B. Replay or Spoofing**

A replay attack occurs when a previously accepted message is captured and transmitted again to make the system process the same data more than once. Spoofing occurs when an unauthorized device creates a message that appears to come from a legitimate sensor. Few related systems experimentally evaluated these threats. Homaei et al. [47] inserted replayed and fabricated sensor readings into a water-consumption dataset and used an LSTM Autoencoder and Isolation Forest to identify them before blockchain storage. However, their tests were conducted at the data-processing layer rather than through actual LoRaWAN packet transmission. A recent LoRaWAN study by Pourghasem et al. [56] also evaluated spoofing detection using device and physical-layer attributes such as RSSI, battery level, and altitude, but its evaluation was dataset-based rather than a live rogue-device attack.

Following the controlled-injection approach of these studies, the proposed prototype will conduct two practical LoRaWAN-level tests: uplink-frame replay and device impersonation through a forged uplink. Testing will be performed only within the isolated experimental network

using the authorized sensor node and a separate programmable LoRa test node. Frame-counter validation will remain enabled in ChirpStack. Each test will include 10 legitimate control transmissions and 10 attack transmissions, producing a total of 40 attempts.

Table 3.8. Replay and Spoofing Test Design

| Test             | Test condition                                                       | Trials | Main measures                                                 | Expected result                                                          | Evidence collected                                               |
|------------------|----------------------------------------------------------------------|--------|---------------------------------------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------|
| LoRaWAN replay   | New legitimate uplink                                                | 10     | Packet acceptance, downstream propagation and processing time | Packet is accepted and reaches the application                           | Gateway, ChirpStack, MQTT and LoRaWAN Analyzer records           |
|                  | Previously accepted uplink retransmitted with an old frame counter   | 10     | Replay rejection and unauthorized propagation                 | Packet is rejected and does not reach MQTT, the database or Fabric       | Gateway frame log, ChirpStack error event and downstream records |
| LoRaWAN spoofing | Genuine uplink from the registered sensor                            | 10     | Packet acceptance, downstream propagation and processing time | Packet is authenticated and processed                                    | Gateway, ChirpStack, MQTT and database records                   |
|                  | Forged uplink using the legitimate device address but an invalid MIC | 10     | Spoofing rejection and unauthorized propagation               | Packet fails authentication and creates no application or storage record | Gateway frame log, MIC error, MQTT, database and Fabric records  |

### LoRaWAN Replay Test

The registered agricultural sensor will first transmit a legitimate uplink containing a synthetic sensor reading. The raw LoRaWAN physical payload, device address, frame counter, frequency, data rate, and time of reception will be recorded from the gateway and ChirpStack frame logs. The legitimate packet must appear as an accepted uplink in ChirpStack and must reach MQTT and Node-RED.

The legitimate sensor will then transmit at least three additional uplinks so that its frame counter advances. Afterward, the previously captured physical payload containing the older frame counter will be retransmitted through the programmable LoRa test node. The test node will use the same AS923 frequency, spreading factor, bandwidth, and coding rate as the captured transmission.

For a trial to be considered valid, the gateway must record that it received the retransmitted radio frame. If the gateway does not receive it, the trial will be repeated because this would represent a transmission failure rather than successful replay rejection. Once received, ChirpStack should identify that the frame counter has already been processed and prevent the frame from producing a device uplink event. ChirpStack documents that a previously seen uplink frame counter produces a retransmission error and is not forwarded as a new application uplink when frame-counter checking is enabled.

After every attempt, the researcher will verify that the replayed message did not appear in:

- the ChirpStack application uplink events;
- the MQTT sensor topic;
- Node-RED's accepted-message flow;
- PostgreSQL/TimescaleDB; or
- the Hyperledger Fabric transaction records.

The procedure will be repeated using 10 newly captured legitimate packets and 10 corresponding replay attempts.

### LoRaWAN Spoofing Test

The spoofing test will simulate a second device attempting to send data using the identity of the legitimate agricultural sensor. The registered sensor will first transmit 10 genuine uplinks as the control condition. Each

transmission must be received by the gateway, authenticated by ChirpStack, forwarded to MQTT, and stored through the normal processing flow.

For the spoofing condition, the programmable LoRa test node will create an uplink containing the current device address of the legitimate sensor and a fabricated sensor value. However, the test node will not possess the legitimate sensor's network session key. Therefore, the message integrity code generated by the test node will not match the legitimate device's credentials.

The forged frame will be transmitted using a valid AS923 radio configuration so that it can be received by the RAK5146 gateway. LoRaWAN uses session keys and a message integrity code to authenticate radio transmissions; when the session key is incorrect, ChirpStack cannot validate the MIC. The gateway may therefore receive the radio signal, but ChirpStack should reject the message before it is published to the application.

Each spoofing trial will be checked at three points:

1. The gateway frame log must confirm that the radio frame was received.
2. ChirpStack must report an authentication or MIC error and must not create an accepted device uplink.
3. MQTT, PostgreSQL/TimescaleDB, and Hyperledger Fabric must contain no record originating from the forged packet.

Ten forged uplinks will be transmitted. A spoofing attempt that is not received by the gateway will not be counted and will be repeated. Figure 3.9 illustrates the LoRaWAN replay and spoofing tests. In the replay test, a previously captured packet with an old frame counter will be retransmitted, while the spoofing test will submit a forged packet using a legitimate device address but an invalid MIC. In both tests, the gateway

must receive the attack frame before ChirpStack rejects it and prevents its propagation to MQTT, Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric.

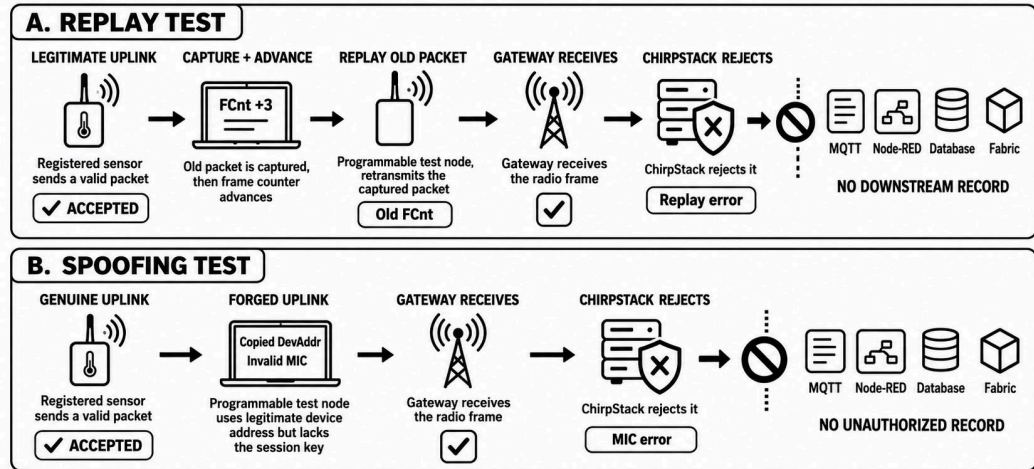


Figure 3.9. LoRaWAN Replay and Spoofing Test Process

## Measures and Data Analysis

For every trial, the researcher will record the gateway reception result, ChirpStack decision, downstream propagation, processing time, and occurrence of an unauthorized database or blockchain record. The following measures will be reported:

- **Replay-rejection rate:** percentage of received replayed frames prevented from reaching the application.
- **Spoofing-rejection rate:** percentage of received forged frames rejected because they could not be authenticated.
- **False-acceptance count:** number of replayed or forged frames incorrectly accepted.
- **False-rejection count:** number of genuine frames incorrectly rejected.
- **Unauthorized propagation rate:** percentage of attack frames that reached MQTT, Node-RED, PostgreSQL/TimescaleDB, or Hyperledger Fabric.

- **Processing time:** time from gateway reception until ChirpStack acceptance or rejection, when matching timestamps are available.

LoRaWAN Analyzer will support the assessment by displaying frame-counter sequences and transmission events, while ChirpStack, MQTT, database, and Fabric logs will provide evidence of whether the packet moved to later layers. ChirpStack's gateway-level frame logging can include frames that fail MIC or frame-counter checks, making it possible to confirm that a packet was received by the gateway but rejected before application processing.

The main security requirement will be zero false acceptance and zero unauthorized propagation. All legitimate control messages should be processed, while no replayed or forged packet should create a sensor record or blockchain transaction.

### C. DoS or Flooding

A Denial-of-Service (DoS) or flooding attack occurs when a large number of requests or messages are sent within a short period to consume processing power, memory, bandwidth, or other system resources. The attack attempts to slow down the system or prevent legitimate sensor readings from being processed. Among the reviewed studies, Hou et al. [40] conducted the clearest DoS-like experiment by generating traffic from authorized and unauthorized devices. Invalid packets were rejected after failing message-integrity verification, while system throughput, CPU utilization, and network bandwidth were measured. Their results showed whether legitimate traffic could continue while invalid traffic increased. The present study will follow the same mixed-traffic approach but adapt it to the MQTT and application layers of the proposed prototype. Radio-frequency jamming will not be included because it requires specialized equipment and may interfere with other wireless users.

Two controlled flooding tests will be conducted: an MQTT connection-flooding test and an invalid application-message flooding test. Both tests will be performed on an isolated local network using a separate laptop as the traffic generator. A legitimate agricultural sensor will continue sending one reading every 15 seconds throughout each test. This will allow the researcher to determine whether valid sensor readings remain available while invalid traffic is being received.

Each test will include three conditions: normal traffic, moderate flooding, and high flooding. Every condition will run for five minutes and will be repeated three times. Therefore, the two tests will involve 18 experimental runs. A five-minute recovery period will be provided after every run to allow the system to clear pending requests and return to normal operation.

Table 3.9. DoS or Flooding Test Design

| Test area                            | Traffic condition | Invalid traffic rate              | Expected result                                                                                       |
|--------------------------------------|-------------------|-----------------------------------|-------------------------------------------------------------------------------------------------------|
| MQTT connection flooding             | Normal traffic    | No invalid attempts               | Legitimate sensor messages are processed normally                                                     |
|                                      | Moderate flooding | 10 invalid connections per second | Invalid connections are rejected while legitimate messages continue                                   |
|                                      | High flooding     | 50 invalid connections per second | System remains available and rejects invalid connections without serious loss of legitimate data      |
| Invalid application-message flooding | Normal traffic    | No invalid messages               | Legitimate messages are validated and stored                                                          |
|                                      | Moderate flooding | 10 invalid messages per second    | Invalid messages are rejected and legitimate messages continue through the system                     |
|                                      | High flooding     | 50 invalid messages per second    | Invalid messages do not create database or blockchain records, and the system recovers after the test |

Note(s): Duration: 5 minutes, Repetitions: 3

The proposed traffic rates will first be checked during a short pilot test. When a selected rate causes an immediate system failure or does not produce a measurable increase in resource use, it may be adjusted before the main experiment. The final traffic rates will remain fixed across all three repetitions.

For all test conditions, one registered sensor will continue transmitting one legitimate reading every 15 seconds, producing approximately 20 valid readings during each five-minute run. Under normal traffic, no invalid requests will be generated. Moderate flooding will introduce 10 invalid connection attempts or messages per second, equivalent to approximately 3,000 invalid requests per run, while high flooding will introduce 50 per second, equivalent to approximately 15,000 invalid requests per run. Each condition will be repeated three times using the same traffic rates to support consistent comparison of system performance.

### **MQTT Connection-Flooding Procedure**

The MQTT connection-flooding test will determine whether repeated connection attempts using incorrect credentials can affect the delivery of legitimate sensor messages. Anonymous MQTT access will remain disabled, and Mosquitto authentication will be enabled.

During the normal condition, the registered agricultural sensor will transmit readings through ChirpStack to Mosquitto MQTT, Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric. No invalid connection attempts will be generated. This condition will establish the normal delivery rate, latency, and resource use of the prototype.

During the moderate and high flooding conditions, a load-generation script on the test laptop will repeatedly attempt to connect to Mosquitto using an incorrect username or password. The script will

generate 10 invalid connection attempts per second during the moderate condition and 50 attempts per second during the high condition. The legitimate sensor will continue transmitting throughout the test.

Mosquitto should reject the invalid connections while continuing to accept messages from ChirpStack and Node-RED. The researcher will examine the Mosquitto logs to count rejected connections and will check Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric to determine whether all legitimate readings were processed.

### **Invalid Application-Message Flooding Procedure**

The invalid application-message flooding test will determine whether Node-RED can reject excessive malformed or unauthorized messages without interrupting legitimate processing. A limited MQTT test account will be allowed to publish only to a controlled test topic connected to the same Node-RED validation flow. The account will not have access to the production ChirpStack topics.

During the normal condition, only legitimate sensor messages will pass through the processing flow. During the flooding conditions, the test laptop will publish invalid JSON messages containing missing fields, incorrect device identifiers, invalid timestamps, or values outside the expected data format. Moderate flooding will generate 10 invalid messages per second, while high flooding will generate 50 invalid messages per second.

Node-RED will validate the incoming messages. Invalid messages should be rejected or placed in a separate test log and must not be stored as valid records in PostgreSQL/TimescaleDB or submitted to Hyperledger Fabric. The legitimate sensor messages should continue to be processed during the flooding period.

After every run, the flood traffic will be stopped. The researcher will continue monitoring the system to determine how long it takes for CPU use, memory use, message processing, and latency to return to their normal levels.

Figure 3.10 illustrates the MQTT connection-flooding and invalid application-message flooding tests under normal, moderate, and high traffic conditions. Throughout each test, the legitimate sensor will continue transmitting one reading every 15 seconds while invalid connections or messages increase from 0 to 10 and 50 per second. The system will be assessed based on its ability to reject invalid traffic, continue processing legitimate readings, prevent unauthorized database or Fabric records, and return to baseline after recovery.

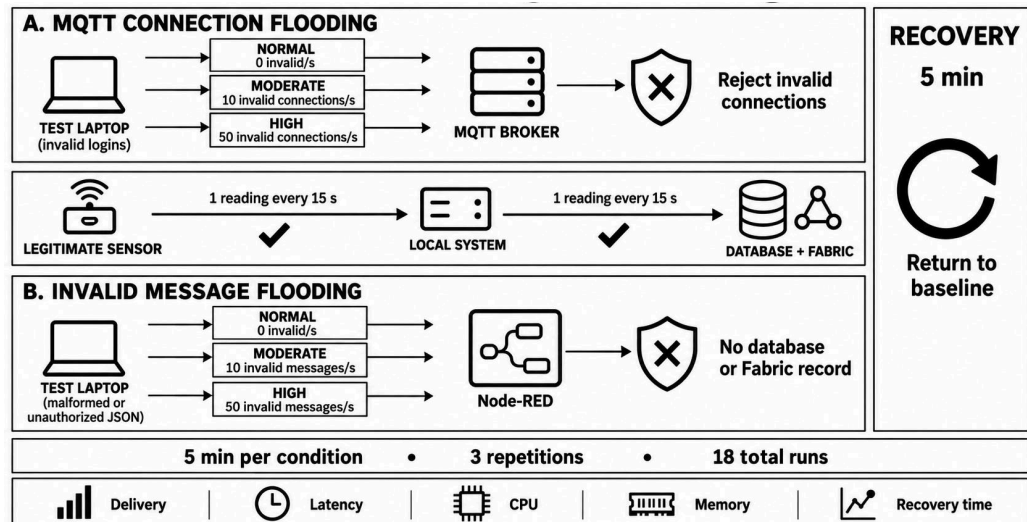


Figure 3.10. DoS or Flooding Test Design

### Measures and Data Analysis

The following measures will be collected during every experimental run:

- **Invalid-traffic rejection rate:** percentage of invalid connections or messages correctly rejected.

- **Legitimate-message delivery rate:** percentage of expected sensor readings successfully stored during the test.
- **End-to-end latency:** time from legitimate sensor transmission until database storage.
- **Valid-message throughput:** number of legitimate sensor readings successfully processed per unit of time.
- **CPU utilization:** average and maximum Raspberry Pi processor use during each condition.
- **Memory utilization:** average and maximum memory use during each condition.
- **Network bandwidth:** amount of network traffic received and transmitted during the test.
- **Service availability:** whether Mosquitto, Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric remained operational.
- **Unauthorized record count:** number of invalid messages incorrectly stored in the database or blockchain.
- **Recovery time:** time from the end of flooding until latency and resource use return to the normal range.

Mosquitto and Node-RED logs will provide the counts of received and rejected requests. PostgreSQL/TimescaleDB and Hyperledger Fabric will be checked for legitimate and unauthorized records. Raspberry Pi CPU, memory, and network utilization will be collected using system and Docker resource-monitoring logs. Grafana may be used to display the measurements throughout each test.

The mean and standard deviation will be reported for latency, throughput, CPU utilization, memory utilization, bandwidth, and recovery time. Frequencies and percentages will be reported for legitimate-message delivery, invalid-traffic rejection, service availability, and unauthorized records. Results from the normal, moderate-flooding, and high-flooding

conditions will be compared to determine whether increasing invalid traffic significantly affects system operation.

The principal security requirements will be that invalid traffic is rejected, no invalid message creates a valid database or blockchain record, and the prototype continues processing legitimate sensor readings. Any service crash, unauthorized record, or sustained failure to process legitimate messages will be classified as a flooding-related security failure.

### **3.2.4.2.3 Data Integrity**

Data integrity refers to maintaining the accuracy and unchanged condition of sensor data while they are processed and stored. Most reviewed studies implemented integrity mechanisms but did not conduct direct tampering experiments. Studies [39], [42], and [44] used IPFS content identifiers or SHA-256 hashes, while Honar Pajooch et al. [43] computed checksums for off-chain IoT data and stored them in Hyperledger Fabric for later verification. Other systems relied on LoRaWAN message integrity codes or blockchain ledger immutability [40], [41], [46], [48]. Homaei et al. [47] conducted a more direct assessment by inserting altered error codes before blockchain recording and modifying blockchain files after storage. The abnormal records were examined before acceptance, while changes made to stored blockchain files were rejected because their hashes no longer agreed with the other validator records.

The present study will adapt the controlled tampering approach of Homaei et al. [47] and the checksum-verification design of Honar Pajooch et al. [43]. Three integrity tests will be conducted: application-layer data alteration, post-storage database tampering, and blockchain-layer duplicate or unauthorized overwrite. The first will determine whether a sensor record changed during Node-RED processing can be detected before it enters the operational database and blockchain. The second will determine whether a PostgreSQL/TimescaleDB record changed after its original hash has been committed to Hyperledger Fabric

can be detected through hash comparison. The third will determine whether an existing blockchain integrity anchor can be duplicated or replaced through a subsequent transaction using the same record identifier.

Testing will be performed on an isolated network using synthetic agricultural sensor readings. Each test will include 10 unchanged control records and 10 deliberately altered records, resulting in 40 total test attempts. Before testing, the PostgreSQL/TimescaleDB database will be backed up, timestamps will be synchronized, and logging will be enabled in Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric.

### **A. Application-layer and post-storage integrity Tests Procedure**

#### **Preparation of the Integrity Record**

For every sensor reading, Node-RED will create a canonical record containing the test identifier, device identifier, frame counter, sensor type, sensor value, measurement unit, and event timestamp. The fields will be placed in a fixed order, timestamps will use the same UTC format, and numerical values will use consistent decimal precision. This standardization is necessary because differences in field order, spacing, or numerical formatting can produce different hashes even when the actual information has not changed.

Node-RED will generate a SHA-256 hash from the canonical record. The complete sensor record will be stored in PostgreSQL/TimescaleDB, while its record identifier, SHA-256 hash, device identifier, and timestamp will be submitted to Hyperledger Fabric. The same canonicalization procedure will be used whenever a stored record is verified.

Table 3.10. Application-Layer Data-Alteration Procedure Test Design

| Test area   | Test condition       | Trials | Main measures | Expected result | Evidence collected |
|-------------|----------------------|--------|---------------|-----------------|--------------------|
| Application | Sensor record passes | 10     | Hash match,   | Initial and     | Node-RED logs,     |

|                        |                                                                                |    |                                                             |                                                             |                                                                                             |
|------------------------|--------------------------------------------------------------------------------|----|-------------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| -layer integrity       | through Node-RED without alteration                                            |    | acceptance decision and verification time                   | processed hashes match; record is accepted                  | database record and Fabric transaction                                                      |
|                        | Sensor value is altered after the initial hash is generated but before storage | 10 | Tamper detection, false acceptance and unauthorized storage | Hash mismatch is detected; record is blocked or quarantined | Original and processed hashes, Node-RED warning and absence of valid database/Fabric record |
| Post-storage integrity | Stored database record remains unchanged                                       | 10 | Hash match, false-positive result and verification time     | Database hash matches the Fabric hash                       | PostgreSQL record, Fabric hash and verification log                                         |
|                        | Stored sensor value is changed after its original hash is committed to Fabric  | 10 | Tamper detection, false negative and detection time         | Recomputed database hash does not match the Fabric hash     | Original Fabric hash, modified database value and integrity alert                           |

### Application-Layer Data-Alteration Procedure

A registered sensor will transmit a synthetic reading through LoRaWAN and ChirpStack to MQTT. When Node-RED receives the message, it will create the canonical record and calculate its initial SHA-256 hash. The record will then pass through the normal validation and processing flow.

For the control condition, the sensor value will remain unchanged. Node-RED will calculate the hash again immediately before database storage. The initial and final processing hashes should match, allowing the record to be stored in PostgreSQL/TimescaleDB and its hash to be submitted to Hyperledger Fabric.

For the alteration condition, a controlled test function in Node-RED will modify one sensor value after the initial hash has been generated. For example, a soil-moisture value of 35% may be changed to 45%. The device identity, frame counter, and timestamp will remain unchanged so that the test represents

unauthorized alteration of the reading rather than the creation of a different record.

Node-RED will calculate a second hash from the altered record and compare it with the initial hash. A mismatch should cause the record to be marked as altered. It will be placed in a separate test or quarantine log and must not be stored as a valid operational reading or submitted to Hyperledger Fabric as a valid sensor transaction. Ten unchanged records and ten altered records will be tested.

### **B. Post-Storage Database-Tampering Procedure**

Ten valid sensor readings will first be processed, stored in PostgreSQL/TimescaleDB, and committed to Hyperledger Fabric using their original SHA-256 hashes. Each record will have a unique test identifier linking the database entry to its corresponding Fabric transaction.

For the control condition, the integrity-verification function will retrieve each unchanged database record, convert it into the defined canonical format, recompute its SHA-256 hash, and compare the result with the hash retrieved from Hyperledger Fabric. All ten unchanged records should produce matching hashes.

For the tampering condition, a temporary database account created only for the controlled security test will be used to change the sensor value of each record after its original hash has been committed to Fabric. The blockchain record will not be changed. Immediately after each modification, the integrity-verification function will be executed.

The function will retrieve the modified PostgreSQL record, recompute its hash, and compare it with the original hash stored in Hyperledger Fabric. Because the sensor value has changed, the hashes should not match. The system should mark the record as altered, produce an integrity warning, and identify its record identifier, device identifier, verification time, database hash, and blockchain hash. After the experiment, the modified records will be restored from the backup.

## Measures and Data Analysis

For every trial, the researcher will record the expected integrity condition, actual verification result, original hash, recomputed hash, verification time, and whether the altered record entered the valid database or blockchain flow. The following measures will be used:

- **Integrity verification rate:** percentage of unchanged records correctly confirmed as valid.
- **Tamper-detection rate:** percentage of altered records correctly identified through a hash mismatch.
- **False-positive count:** unchanged records incorrectly identified as altered.
- **False-negative count:** altered records incorrectly reported as valid.
- **Unauthorized storage count:** altered application-layer records stored as valid operational records.
- **Verification time:** time from the start of the hash comparison until the system reports a match or mismatch.

Frequencies and percentages will be used to report integrity decisions. Verification time will be summarized using the mean and standard deviation. Results will be reported separately for application-layer alteration and post-storage tampering because the first test examines protection before valid storage, while the second examines verification after storage.

The security requirement will be zero false negatives and zero unauthorized storage. Every unchanged record should produce a matching hash, while every deliberately altered record should produce a hash mismatch. The test verifies whether data remain unchanged after the first hash is generated; it does not prove that the original sensor reading was physically accurate before entering the system.

### C. Blockchain-layer integrity Tests Procedure

The blockchain-layer integrity test will determine whether an existing sensor-record hash stored in Hyperledger Fabric can be duplicated or replaced through a subsequent request using the same record identifier. The procedure is adapted from Liu et al. [62], whose Hyperledger Fabric chaincode checks whether a unique vehicle identifier already exists before creating a new ledger record and returns an error when the identifier is reused. In the present study, this principle will be applied to sensor-data integrity records by treating each `trace_id` as a unique key associated with the SHA-256 hash of one sensor record. Unlike applications in which an existing blockchain state may legitimately be updated, the sensor hash in this prototype will function as a create-only integrity anchor and therefore must not be replaced after its initial commitment.

This additional test is necessary because Hyperledger Fabric distinguishes the immutable blockchain transaction log from the world state, which represents the latest value associated with each ledger key. A world-state value may be changed through a valid transaction when the chaincode permits an update. Accordingly, the chaincode used for the sensor-hash records will first determine whether the submitted `trace_id` already exists. A new hash record will be created only when the identifier does not yet exist. If the same identifier is submitted again, the request will be rejected and the previously stored hash must remain unchanged. This follows the create-before-existence-check pattern demonstrated by Liu et al. [62] and by Hyperledger Fabric's chaincode implementation guidance.

Ten valid sensor records will first be processed using the normal system flow. Each record will have a unique `trace_id`, device identifier, timestamp, and SHA-256 hash. The ten records will be submitted to Hyperledger Fabric and confirmed as successfully committed. The Fabric world state will then be queried to verify that the correct hash is associated with each `trace_id`. These records will serve as the baseline for the duplicate and overwrite conditions.

For the duplicate condition, each of the ten previously committed records will be submitted again using the same `trace_id` and the same SHA-256 hash. Because the identifier already exists, the chaincode should reject each second create request. The purpose of this condition is to determine whether duplicate blockchain anchors can be created when the same valid record is resubmitted, such as through an application retry or duplicate-processing event.

For the unauthorized-overwrite condition, each of the same ten `trace_id` values will again be submitted, but the original SHA-256 hash will be replaced with a deliberately different hash. The request will be made using an otherwise authorized Fabric identity. This condition represents a compromised or improperly functioning authorized application attempting to associate a different hash with an already established sensor-record identifier. Although the submitting identity is valid, the requested operation is unauthorized because an existing integrity anchor should not be replaceable.

After every duplicate and overwrite attempt, the corresponding Fabric world-state entry will be queried and compared with the original hash. The original hash associated with each `trace_id` must remain unchanged. The application and Fabric logs will also be examined for the chaincode response, transaction or proposal identifier where available, validation result, submitted hash, original hash, and decision timestamp. Because a request rejected during chaincode execution may not become a committed blockchain transaction, rejection evidence will be obtained primarily from the Fabric client, gateway, peer, or chaincode logs, while ledger queries will be used to confirm that no valid state change occurred.

The **duplicate-rejection rate** will represent the percentage of duplicate submissions correctly rejected and will be calculated using equation (31):

$$\text{Duplicate-Rejection Rate} = \frac{N_{\text{duplicates rejected}}}{N_{\text{duplicate attempts}}} \times 100 \quad (31)$$

where:  $N_{\text{duplicates rejected}}$  = number of duplicate submissions correctly rejected by the system; and  $N_{\text{duplicate attempts}}$  = total number of duplicate submission attempts.

The **overwrite-rejection rate** will represent the percentage of conflicting submissions using an existing trace\_id and a different hash that are correctly rejected and will be calculated using equation (32):

$$\text{Overwrite-Rejection Rate} = \frac{N_{\text{overwrites rejected}}}{N_{\text{overwrite attempts}}} \times 100 \quad (32)$$

where:  $N_{\text{overwrites rejected}}$  = number of conflicting submissions using an existing trace\_id and a different hash that are correctly rejected; and  $N_{\text{overwrite attempts}}$  = total number of conflicting overwrite attempts submitted to the system.

The **original-hash preservation rate** will determine whether the blockchain integrity anchors remain unchanged following the test attempts and will be calculated using equation (33):

$$\text{Original-Hash Preservation Rate} = \frac{N_{\text{original hashes unchanged}}}{N_{\text{blockchain records tested}}} \times 100 \quad (33)$$

where:  $N_{\text{original hashes unchanged}}$  = number of blockchain records whose original hashes remain unchanged after the test attempts; and  $N_{\text{blockchain records tested}}$  = total number of blockchain records included in the test.

The number of unauthorized state changes will also be recorded. An unauthorized state change will occur if a duplicate or conflicting submission causes the hash associated with an existing trace\_id to be replaced or otherwise modified in the Fabric world state.

#### 3.2.4.2.4 Traceability

Traceability refers to the ability of the prototype to identify the origin and complete processing history of every sensor reading. A traceable record should show which device produced the reading, when it was transmitted and processed, where the complete record was stored, and which Hyperledger Fabric transaction contains its hash and metadata. Previous studies commonly demonstrated traceability through blockchain histories and record queries rather than through direct attack experiments. Keo et al. [42] recorded the movement and history of

agricultural batches, Honar Pajooch et al. [43] stored information about data ownership, submission time, storage location, and related records, while Yang et al. [44] used unique codes to retrieve complete product histories. Other studies used Fabric historian records, blockchain audit logs, and transaction histories to trace sensor readings and system activities [46]–[48]. However, most did not report a separate traceability success rate or evaluate whether every stored record could be completely and correctly reconstructed.

The present study will evaluate traceability through two tests: an individual sensor-record trace test and a device-history reconstruction test. The first will determine whether one sensor reading can be followed from its LoRaWAN source to PostgreSQL/TimescaleDB and Hyperledger Fabric. The second will determine whether a sequence of readings from one device can be retrieved in the correct chronological order with complete links between the operational database and blockchain records.

Testing will be conducted on the isolated prototype network using synthetic agricultural sensor readings. The individual-record test will use 10 readings, while the device-history reconstruction test will use 10 sequences containing five consecutive readings each. This will produce 20 traceability trials involving 60 sensor records. Logging will be enabled in ChirpStack, Mosquitto MQTT, Node-RED, PostgreSQL/TimescaleDB, and Hyperledger Fabric.

### **Preparation of Traceability Records**

Node-RED will assign a unique trace\_id to every accepted sensor reading. The trace identifier will remain attached to the record as it passes through the processing, storage, and blockchain-verification layers. The following information will be retained:

- trace identifier;
- device EUI;
- LoRaWAN frame counter;

- sensor type, value, and unit;
- sensor or event timestamp;
- gateway identifier;
- PostgreSQL record identifier;
- SHA-256 hash;
- Hyperledger Fabric transaction identifier; and
- blockchain commit timestamp or block number.

The complete sensor reading and related operational information will be stored in PostgreSQL/TimescaleDB. The trace identifier, hash, device identity, timestamp, and essential metadata will be recorded in Hyperledger Fabric. The common trace identifier will allow the database record to be linked directly to its blockchain transaction.

Table 3.11. Traceability Test Design

| Test area                      | Test condition                                                                                   | Trials       | Main measures                                                                                         | Expected result                                                                     | Evidence collected                                                                      |
|--------------------------------|--------------------------------------------------------------------------------------------------|--------------|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Individual sensor-record trace | Retrieve one selected reading from its LoRaWAN source through the database and blockchain record | 10           | Retrieval success, field completeness, database–blockchain linkage, and retrieval time                | Every reading is located with complete and correctly linked information             | ChirpStack event, Node-RED log, PostgreSQL record, Fabric transaction, and hash         |
| Device-history reconstruction  | Retrieve five consecutive readings from one device and reconstruct their processing history      | 10 sequences | Chronological accuracy, missing records, duplicate records, linkage accuracy, and reconstruction time | All readings are returned in the correct order without missing or duplicate records | PostgreSQL query results, Fabric history query, transaction IDs, timestamps, and hashes |

#### A. Individual Sensor-Record Trace Procedure

The registered sensor will transmit 10 synthetic readings through LoRaWAN. Each reading will contain a known test value and will be assigned a unique trace identifier when received by Node-RED. The researcher will record

the device EUI, frame counter, event timestamp, sensor value, and gateway information from ChirpStack and Node-RED.

After the reading has been processed, the researcher will use the trace identifier to retrieve its complete record from PostgreSQL/TimescaleDB. The corresponding Hyperledger Fabric transaction will then be retrieved using the transaction identifier stored with the database record.

For each trial, the fields recorded at the LoRaWAN and application layers will be compared with the database and blockchain information. The test will determine whether the device identity, frame counter, sensor value, timestamp, record identifier, hash, and Fabric transaction are available and correctly connected. A trial will be considered successful only when the complete path of the reading can be reconstructed without an incorrect or missing link.

## **B. Device-History Reconstruction Procedure**

The registered sensor will produce 10 separate sequences, with each sequence containing five consecutive readings transmitted at fixed intervals. Each record will receive a unique trace identifier and will be stored in PostgreSQL/TimescaleDB with its corresponding Fabric transaction identifier.

For every sequence, the researcher will query PostgreSQL/TimescaleDB using the device EUI and defined time range. The retrieved records will be arranged according to their timestamps and frame counters. The related blockchain records will then be obtained through their Fabric transaction identifiers or chaincode history query.

The reconstructed sequence will be compared with the original transmission log. The researcher will verify whether all five readings are present, appear in the correct order, contain the correct values and timestamps, and remain linked to their corresponding blockchain hashes and transactions. Missing, duplicate, incorrectly ordered, or incorrectly linked records will be recorded as traceability errors.

## Measures and Data Analysis

For every trial, the researcher will record whether the expected records were retrieved, whether the required fields were complete, whether the database and blockchain records were correctly linked, and how long the retrieval or reconstruction required. The following measures will be used:

- **Trace retrieval success rate:** percentage of individual records successfully retrieved from both PostgreSQL/TimescaleDB and Hyperledger Fabric.
- **Record completeness rate:** percentage of required traceability fields available and correct.
- **Database–blockchain linkage rate:** percentage of database records correctly connected to their corresponding blockchain hashes and transaction identifiers.
- **Chronological-order accuracy:** percentage of readings reconstructed in the correct sequence.
- **Missing-record count:** number of transmitted readings that could not be retrieved.
- **Duplicate-record count:** number of readings appearing more than once in the reconstructed history.
- **Trace retrieval time:** time required to retrieve the complete history of an individual record.
- **History reconstruction time:** time required to retrieve and arrange one sequence of device readings.

Frequencies and percentages will be used to summarize trace retrieval, completeness, linkage, chronological order, and missing or duplicate records. Retrieval and reconstruction times will be reported using the mean and standard deviation. The two tests will be reported separately because the individual-record test examines the traceability of one reading, while the history-reconstruction test evaluates the continuity of several readings over time.

The traceability requirement will be complete retrieval and correct linkage of all tested records. Every sensor reading should be connected to its original

device information, PostgreSQL record, SHA-256 hash, and Hyperledger Fabric transaction. No expected record should be missing, duplicated, or placed in the wrong chronological order.

### **3.2.5 Revision**

The Revision phase will address technical issues identified during prototype integration, pilot testing, and formal evaluation. Because the architecture will combine communication, edge processing, time-series storage, blockchain verification recording, security controls, and recovery mechanisms, adjustments may be required to improve compatibility, synchronization, reliability, and resilience across components. These may include refining configurations and Node-RED flows, improving database–blockchain synchronization, strengthening access controls, and modifying recovery procedures. Affected components and tests will be revalidated after each change, while unresolved technical limitations will be documented for future development.

## **3.3 Ethical Consideration**

This study will involve the development and controlled testing of an IoT- and blockchain-based prototype and will not collect data from human participants. The sensors will gather only soil and environmental readings, such as temperature and humidity, without collecting personal, medical, financial, or identifying information. Permission will be obtained from the plantation management before installation and field testing, and the activities will be conducted without disrupting normal farm operations.

Device identifiers, access keys, credentials, server addresses, and blockchain identities will be securely stored and restricted to authorized users. Sensor records will be used only for prototype development, verification, and evaluation and will be retained and disposed of according to applicable institutional requirements. Simulated attacks, outages, and record modifications will be conducted only within an isolated and controlled test environment.

Backups will be maintained, and no operational plantation records, external systems, or unauthorized devices will be affected.

Results will be reported accurately, including successful and failed tests, technical limitations, and unexpected outcomes. The earlier publication on which this dissertation builds will be properly cited, and reused methods, system elements, data, or findings will be disclosed to avoid misrepresentation or duplicate publication. The study will also comply with applicable institutional research-ethics and information-security requirements.